
A TRAINING-FREE MIXTURE-OF-AGENTS FRAMEWORK FOR MULTI-DOCUMENT SUMMARIZATION USING LLMs AND KNOWLEDGE GRAPHS

Cuong Vuong Tuan¹, Trang Mai Xuan^{1*}, Tien-Cuong Nguyen², Vu-Duc Ngo³, Thien Van Luong⁴

¹Faculty of Artificial Intelligence and Data Science, Phenikaa University, Duong Noi, Ha Noi, Viet Nam

²VNPT AI, VNPT Group, Ha Noi, Viet Nam

³MobiFone Research and Development Center, MobiFone Corporation, Ha Noi, Viet Nam

⁴Business AI Lab, Faculty of Data Science and Artificial Intelligence, National Economics University, College of Technology, Ha Noi, Viet Nam

cuong.vuongtuan@phenikaa-uni.edu.vn, trang.maixuan@phenikaa-uni.edu.vn,
nguyentiencong@vnpt.vn, duc.ngo@mobifone.vn, thienlv@neu.edu.vn

June 24, 2026

Comments: The Version of Record of this article is published in *Neural Computing and Applications*, and is available online at <https://doi.org/10.1007/s00521-026-12249-y>.

Abstract

Multi-Document Summarization (MDS) play a critical role in distilling essential information from collections of textual data. Existing approaches often struggle to capture complex inter-document relationships, rely heavily on large amounts of labeled data for supervised training, or exhibit limited generalization across domains and languages. To address these limitations, we present a training-free mixture-of-agents framework for MDS that leverages the complementary strengths of large language models (LLMs) and knowledge graphs. Our approach decomposes summarization into specialized agent tasks: extractive selection, knowledge-aware abstraction, and iterative refinement, each operating without task-specific fine-tuning. We unify their outputs using a multi-perspective consistency mechanism guided by LLMs. Experiments across four datasets in English and Vietnamese demonstrate state-of-the-art or competitive performance, validating the effectiveness and adaptability of our modular design.

Keywords Multi-Document Summarization, Large Language Models, Mixture of Agents, Knowledge Graphs

1 Introduction

Rapidly expanding digital information necessitates MDS for synthesizing knowledge from diverse sources, making it a critical task in natural language processing [Mridha et al., 2021, Ma et al., 2022]. Early extractive methods [Erkan and Radev, 2004, Mihalcea and Tarau, 2004] and rule-based abstractive systems [Wang et al., 2017] often produce outputs that lack semantic coherence and fluency. The advent of deep learning, especially sequence-to-sequence architectures [Sutskever et al., 2014] and subsequently transformer-based

*Corresponding author: trang.maixuan@phenikaa-uni.edu.vn

LLMs [Vaswani et al., 2017, Wu et al., 2025], has significantly improved the quality of abstractive summarization [Zhang et al., 2020, Xiao et al., 2022]. However, leading supervised models require large task-specific labeled datasets [Fabbri et al., 2019, Lu et al., 2020], limiting their applicability in low-resource domains or under-resourced languages such as Vietnamese [Tran et al., 2020].

Recent advances in LLMs have opened new opportunities for MDS through strong zero-shot and few-shot capabilities [Brown et al., 2020, Wei et al., 2022]. However, context length limitations remain a significant barrier. Moreover, The et al. [Le and Le, 2025] conducted specific fine-tuning on Vietnamese datasets, achieving promising results. Nevertheless, these approaches face challenges including limited testing on diverse English datasets and requirements for extensive data collection and labeling. Furthermore, current training-free methods that use pre-trained LLMs encounter specific difficulties in learning complex inter-document relationships and handling large semantic contexts [Shakil et al., 2024]. Therefore, specialized frameworks are needed that can use existing LLM knowledge without requiring task-specific data collection or fine-tuning, while addressing coordination challenges across multiple documents.

This paper introduces the **Mixture of Agents (MoA)** framework, a novel architectural approach for MDS that leverages pre-trained LLMs within a collaborative multi-agent system **without task-specific fine-tuning**. The MoA architecture decomposes the complex MDS task into complementary sub-problems managed by specialized agents as follows:

- (1) **Extractor agent:** Identifies salient factual sentences using a robust scoring model.
- (2) **KGSum agent:** Our most sophisticated agent, which constructs a knowledge graph (KG) to model entities and their relationships explicitly. It then generates a KG-based summary by identifying thematic communities and detecting contrastive information within the graph, offering a deep, structured perspective.
- (3) **Abstractor agent:** Produces a fluent, coherent abstractive summary directly from the source documents.

The outputs from these parallel agents are orchestrated by our novel **Adaptive Multi-Perspective Fusion (AMF)** mechanism. AMF assesses agent-generated metadata to dynamically select the optimal strategy for synthesizing a final summary, robustly fusing extractive, KG-based, and abstractive perspectives.

Our primary contributions are as follows:

- We propose MoA, a novel architectural framework for training-free multi-document summarization that coordinates three specialized agents: Extractor, KGSum and Abstractor. The main innovation is the KGSum agent that constructs knowledge graphs to explicitly model complex inter-document relationships and contradictions, enabling systematic learning of intricate connections between multiple documents.
- We introduce the Adaptive Multi-Perspective Fusion (AMF) mechanism that uses agent-generated metadata to dynamically determine optimal fusion strategies, allowing MoA to adaptively exploit agent-specific strengths based on document characteristics.
- We demonstrate MoA’s effectiveness through comprehensive evaluations using various LLMs on benchmark datasets across English and Vietnamese, achieving state-of-the-art performance and validating our architectural approach for learning complex inter-document relationships.

The remainder of the paper is organized as follows: Section 2 discusses related work in MDS. Section 3 details the MoA framework. Section 4 presents the experimental setup, results, and analysis. Section 5 concludes and suggests future directions.

2 Related Work

2.1 Training-free Approaches for MDS

Early efforts in this paradigm were dominated by traditional unsupervised extractive techniques. For example, maximal marginal relevance, which selects relevant yet diverse sentences, was proposed in [Goldstein and Carbonell, 1998], while centroid-based approaches that identify sentences central to a document cluster’s theme were introduced in [Radev et al., 2004]. Additionally, graph-based algorithms such as LexRank [Erkan and Radev, 2004] and TextRank [Mihalcea and Tarau, 2004] rely on inter-sentence relationships to

rank sentences by centrality. Early hybrid extractive-abstractive methods combined extractive ranking with abstractive generation [Wang et al., 2017].

More recent unsupervised approaches have incorporated deep learning representations without task-specific fine-tuning. Examples include using pre-trained model embeddings within graph-based frameworks [Li et al., 2020] or leveraging topic modeling with transformer representations, as seen in GLIMMER-TTR [Liu et al., 2024]. Unsupervised graph-based methods were also explored for languages such as Vietnamese [Doan-Thanh et al., 2023].

The advent of LLMs has further expanded training-free MDS possibilities, primarily through zero-shot or few-shot prompting [Shakil et al., 2024]. Li et al. [Li et al., 2023] focused on strategies to compress or select information before presenting concatenated documents to an LLM due to context window limitations. While these LLM-driven prompting approaches offer flexibility and reduce the need for specific MDS training datasets, directly prompting a single LLM for complex MDS can struggle with information reconciliation and lack coherence across numerous documents.

2.2 Full-training approaches for MDS

Contrasting with training-free methods, full training approaches involve extensive supervised learning on large-scale, task-specific MDS datasets. The advent of deep learning, particularly sequence-to-sequence models [Sutskever et al., 2014], significantly advanced abstractive summarization. For example, Pointer-Generator Networks [See et al., 2017] was proposed to improve the factual consistency.

However, the paradigm shifted decisively with PLMs fine-tuned specifically for MDS. Architectures designed for long sequences (e.g., Longformer [Beltagy et al., 2020]) were adapted. End-to-end supervised training on large MDS datasets, such as Multi-News [Fabbri et al., 2019] and Multi-XScience [Lu et al., 2020] for English, or VN-MDS and ViMs [Tran et al., 2020] for Vietnamese, led to powerful models, such as PEGASUS [Zhang et al., 2020] (gap-sentence objective), PRIMERA [Xiao et al., 2022] (entity-masking), and Centrum [Puduppully et al., 2023] (discourse coherence). Some research works also explored fine-tuning general-purpose LLMs on MDS tasks, which falls under the full training paradigm due to their reliance on task-specific labeled data.

These models typically achieve SOTA ROUGE scores [Lin, 2004] on in-domain data but are fundamentally dependent on such labeled datasets. This restricts their applicability in low-resource scenarios, novel domains, or for languages whose large specific MDS corpora are unavailable. MoA, as a training-free approach, aims to circumvent this data dependency entirely.

2.3 Agent-Based Systems

The emergence of knowledge-enhanced retrieval systems has shown promise for various AI tasks. Retrieval-Augmented Generation (RAG) [Lewis et al., 2020] represents a major advance in knowledge-intensive tasks, showing strong results in question-answering by combining retrieval with text generation. GraphRAG [Edge et al., 2024] extends this approach using graph structures to capture relationships and enable better reasoning. However, these query-focused methods may not suit the multi-perspective analysis needed for comprehensive document summarization.

Recently, multi-agent systems powered by LLMs have gained significant attention for complex problem-solving and collaboration [Park et al., 2023, Wang et al., 2024]. AutoGen [Wu et al., 2023] enables multiple agents to converse and complete tasks through flexible interaction patterns. Similarly, ChatEval [Chan et al., 2024] uses multi-agent debates where LLMs collaborate as evaluators to enhance task performance. Moreover, the Mixture-of-Agents approach [Wang et al., 2025] constructs layered architectures where each agent leverages outputs from previous layer agents as auxiliary information to enhance response quality. While these frameworks excel in their respective domains, MDS presents unique challenges that benefit from parallel processing of diverse summarization perspectives rather than iterative debate refinement.

To the best of our knowledge, this work presents the first attempt at developing a structured multi-agent MoA architecture specifically for MDS. Our key contribution lies in integrating AMF mechanisms within the MoA paradigm for training-free MDS, differentiating our approach from monolithic LLM prompting and traditional MDS techniques.

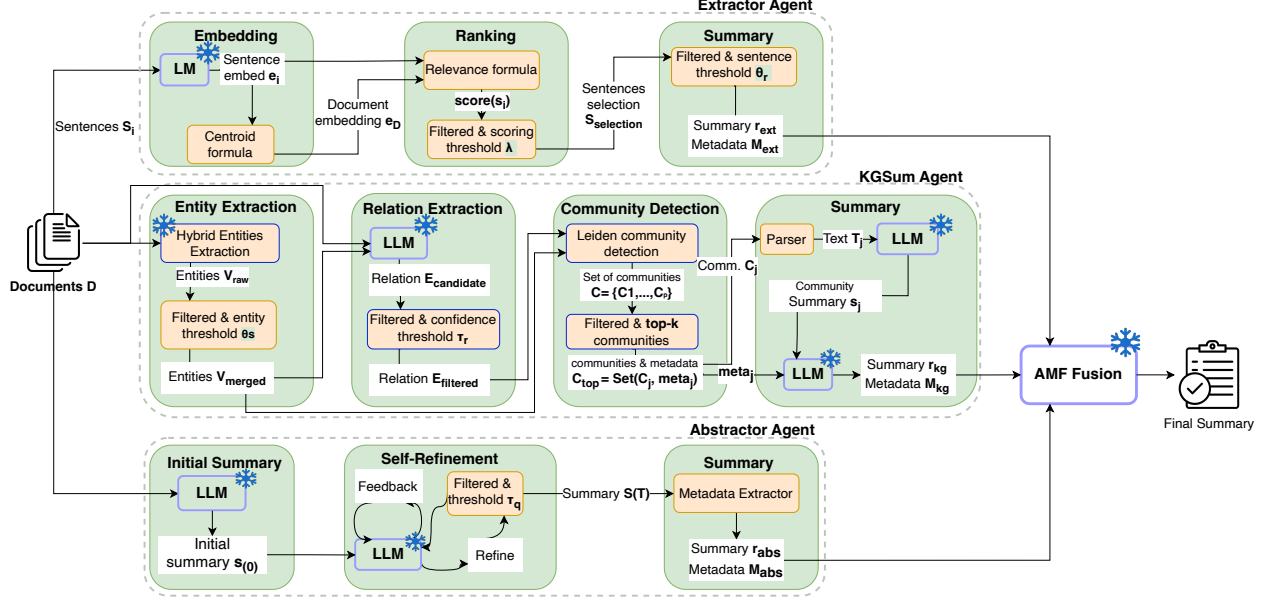


Figure 1: Overall architecture of our proposed MoA framework. The three agents (Extractor, KGSUM, Abstractor) process the input documents D through parallel specialized pipelines, generating their respective summaries (r_{ext} , r_{kg} , r_{abs}) and metadata (M_{ext} , M_{kg} , M_{abs}). The AMF module then synthesizes these outputs into the final summary S_{MoA} . “LM” denotes Language Model (BERT-based), while “LLM” denotes Large Language Model.

3 Methodology

Figure 1 shows the overall framework architecture of MoA framework.

3.1 Extractor Agent

The Extractor Agent systematically identifies and extracts the most semantically significant sentences from the input document set D to generate an extractive summary r_{ext} . Building upon established centroid-based approaches [Rossiello et al., 2017], our innovation integrates positional awareness with semantic centrality through a hybrid scoring mechanism that leverages pre-trained contextual embeddings. As illustrated in the Extractor Agent section of Figure 1, this process operates through three sequential stages: embedding generation, relevance scoring, and summary assembly.

In the first stage, neural sentence embedding transforms each sentence s_i from document set D into contextual vector representations using a pre-trained Language Model (LM), specifically Longformer [Beltagy et al., 2020]. This encoding process produces sentence embeddings e_i that capture semantic content within the document context. Subsequently, the document-level semantic representation e_D is computed as the centroid of all sentence embeddings:

$$e_D = \frac{1}{m} \sum_{i=1}^m e_i, \quad (1)$$

where m denotes the total number of sentences across all documents in D .

The second stage implements sentence ranking through a hybrid relevance scoring mechanism that balances semantic similarity with positional bias. For each sentence s_i , the relevance score integrates cosine similarity between sentence and document embeddings with positional preference:

$$\text{score}(s_i) = \lambda \cos(e_i, e_D) + (1 - \lambda) \left(1 - \frac{\text{pos}(s_i)}{L_d} \right), \quad (2)$$

where $\text{pos}(s_i)$ represents the normalized position of sentence s_i within the concatenated document set of length L_d , and λ controls the semantic-positional balance, as shown in Table 13. Sentences are subsequently ranked by these scores, with the top- α percent forming the candidate set S_{cand} .

The third stage assembles the final extractive summary through redundancy filtering and metadata generation. An optional redundancy removal step refines S_{cand} into S_{sel} :

$$S_{\text{sel}} = \{s_i \in S_{\text{cand}} : \forall s_j \in S_{\text{sel}}, \cos(\mathbf{e}_i, \mathbf{e}_j) \leq \theta_r\}, \quad (3)$$

where $\mathbf{e}_i$ and $\mathbf{e}_j$ are the embeddings of sentences s_i and s_j , respectively. And θ_r denotes the redundancy threshold (Table 13). Finally, the final summary r_{ext} concatenates selected sentences in their original document order, while the metadata package $\mathcal{M}_{\text{ext}}$ encapsulates sentence scores, selection indices, and processing parameters for subsequent AMF integration.

3.2 KGSum Agent

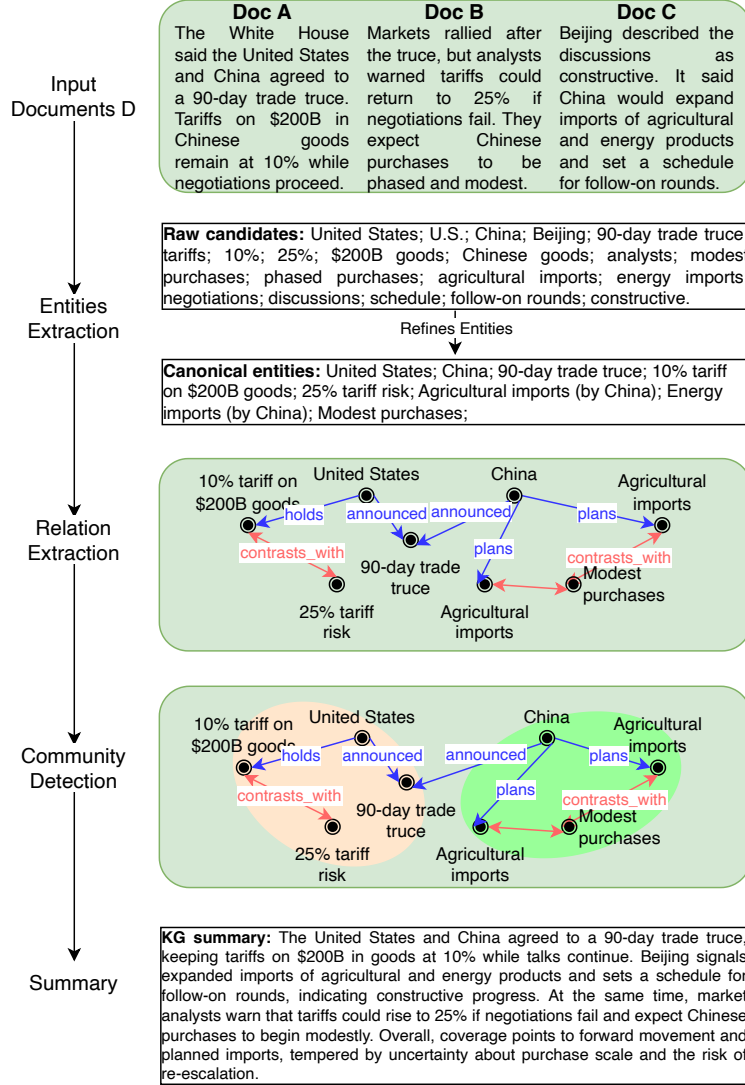


Figure 2: Process of KGSum Agent for dynamic knowledge graph construction and knowledge-enhanced summary generation. Blue relation edges represent standard semantic relations, while red edges denote contrastive relations capturing conflicting information. The knowledge graph is dynamically constructed for each input document set, ensuring tailored representation of the specific content.

The KGSum Agent dynamically constructs a knowledge graph from documents D to generate knowledge-enhanced summaries r_{kg} . Building upon established KG construction principles [Hogan et al., 2021], our innovation extends traditional approaches with hybrid entity extraction, contrastive relation modeling, and

meta-aware summarization, with detailed novelty analysis provided in Appendix B. The dynamic construction ensures that each new document set or revised data generates a fresh, tailored knowledge graph optimized for the specific input. Figure 1 shows the overall architecture, while Figure 2 demonstrates the processing pipeline. The agent operates through four sequential stages, each building upon the previous stage’s outputs.

3.2.1 Entity Extraction from Documents

From the document collection D , the first stage employs hybrid entity extraction to identify key information units, as illustrated in the entity extraction phase of Figure 2. We combine three complementary methods: spaCy NER for standard entities, rule-based patterns for numerical data, and LLM extraction with prompt Π_e (Figure 5) for contextual understanding. The raw entities are aggregated as:

$$V_{\text{raw}} = V_{\text{spacy}} \cup V_{\text{rules}} \cup V_{\text{llm}}. \quad (4)$$

Subsequently, entity consolidation eliminates redundancy through similarity-based merging, as demonstrated in the entity refinement step of Figure 2. For entities $v_i, v_j \in V_{\text{raw}}$, the merging process follows:

$$V_{\text{merged}} = \begin{cases} \text{extMerge}(v_i, v_j) & \text{if } \text{sim}(v_i, v_j) \geq \theta_s \\ \{v_i, v_j\} & \text{otherwise} \end{cases}, \quad (5)$$

where θ_s represents the similarity threshold (Table 13). This process yields the consolidated entity set V_{merged} for subsequent relation extraction.

3.2.2 Relation Extraction from Documents

Leveraging the consolidated entities V_{merged} , the second stage identifies semantic connections to construct graph edges, as shown in the relation extraction phase of Figure 2. Our approach focuses on extracting both standard semantic relations and contrastive relations that capture conflicting information across documents. Using LLM with specialized prompt Π_{rel} (Figure 6), the relation extraction process is formulated as:

$$E_{\text{candidate}} = \text{LLM}(\Pi_{\text{rel}}, V_{\text{merged}}, D). \quad (6)$$

The prompt systematically guides extraction of traditional semantic relations alongside our novel contrastive relations, with empirical validation of contrastive relation effectiveness detailed in Figure 6. To ensure high-quality graph construction, confidence-based filtering is applied to each candidate relation $r \in E_{\text{candidate}}$:

$$E_{\text{filtered}} = \begin{cases} r & \text{if } \text{confidence}(r) \geq \tau_r \\ \emptyset & \text{otherwise} \end{cases}, \quad (7)$$

where τ_r represents the confidence threshold (Table 13). This yields the refined relation set E_{filtered} that forms the knowledge graph edges.

3.2.3 Community Detection in Knowledge Graph

With the knowledge graph $G = (V_{\text{merged}}, E_{\text{filtered}})$ constructed, the third stage partitions it into thematic communities for structured analysis, as illustrated in the community detection phase of Figure 2. We employ the Leiden algorithm [Traag et al., 2019] for community detection based on modularity optimization. The partitioning process produces:

$$\mathcal{C} = \{C_1, C_2, \dots, C_p\} = \text{Leiden}(G). \quad (8)$$

Subsequently, metadata extraction generates descriptive information for each community C_j :

$$\text{meta}_j = \{\text{centrality}, \text{contrast_flags}, \text{key_entities}, \text{fact_density}\}. \quad (9)$$

This metadata encompasses centrality scores, contrastive relation indicators, key entities, and content density measures. Finally, importance-based ranking selects the top- k communities (Table 13) to form $\mathcal{C}_{\text{top}}$ for focused summarization.

3.2.4 KG-Based Summary Generation

Building upon the selected communities $\mathcal{C}_{\text{top}}$, the final stage generates the knowledge-enhanced summary r_{kg} through meta-aware fusion, as demonstrated in the summary generation phase of Figure 2. Each community $C_j \in \mathcal{C}_{\text{top}}$ undergoes linearization and targeted summarization:

$$T_j = \text{Linearize}(C_j), \quad s_j = \text{LLM}(\Pi_s, T_j), \quad (10)$$

where T_j converts the graph structure to text, and Π_s (Figure 7) generates community-specific summaries. Subsequently, meta-aware fusion integrates individual summaries while leveraging their associated metadata:

$$r_{\text{kg}} = \begin{cases} \text{extLLM}(\Pi_f, \{(s_j, \text{meta}_j) : C_j \in \mathcal{C}_{\text{top}}\}) & \text{if } |\mathcal{C}_{\text{top}}| > 1 \\ s_1 & \text{if } |\mathcal{C}_{\text{top}}| = 1 \end{cases}, \quad (11)$$

The fusion prompt Π_f (Figure 8) utilizes metadata to weight communities by centrality, highlight conflicts, and ensure comprehensive coverage. This process yields the final knowledge-enhanced summary r_{kg} along with metadata package $\mathcal{M}_{\text{kg}}$ for subsequent AMF integration.

3.3 Abstractor Agent

The Abstractor Agent generates fluent, abstractive summaries by synthesizing information from documents D to produce r_{abs} . Building upon established LLM summarization techniques [Zhang et al., 2024] and self-improvement concepts [Madaan et al., 2023], our innovation introduces an iterative self-refinement loop specifically designed for multi-document summarization. As illustrated in the Abstractor Agent section of Figure 1, this process operates through three sequential stages: initial summary generation, self-refinement, and final output production.

In the first stage, initial summary generation transforms the document collection D into an abstractive summary using an LLM with specialized prompt Π_{init} (Figure 9). This process generates the initial summary:

$$\tilde{s}^{(0)} = \text{LLM}(\Pi_{\text{init}}, D), \quad (12)$$

where $\tilde{s}^{(0)}$ captures the main themes from D and serves as the foundation for subsequent refinement.

The second stage implements iterative self-refinement to enhance summary quality through feedback-driven improvement. For each iteration t , the system evaluates the current summary $\tilde{s}^{(t)}$ using a multi-dimensional feedback function $\mathcal{F}_{\text{MDS}}$ that assesses factual accuracy, coverage, coherence, and specificity. The refinement process is formulated as:

$$\tilde{s}^{(t+1)} = \text{LLM}(\Pi_{\text{ref}}, \tilde{s}^{(t)}, \mathcal{F}_{\text{MDS}}(\tilde{s}^{(t)}, D)), \quad (13)$$

where Π_{ref} represents the refinement prompt (Figure 10) that guides quality improvement. This iterative cycle continues for a maximum of T iterations or until the quality score exceeds threshold τ_q (Table 13).

The third stage produces the final abstractive summary and comprehensive metadata. The refined summary becomes the final output $r_{\text{abs}} = \tilde{s}^{(T)}$, while the metadata package $\mathcal{M}_{\text{abs}}$ encapsulates quality scores, iteration count, and processing parameters. Both r_{abs} and $\mathcal{M}_{\text{abs}}$ are subsequently passed to the AMF module for multi-perspective integration.

3.4 Mixture of Agents Orchestration: The Adaptive Multi-Perspective Fusion (AMF) Mechanism

The three agents produce specialized summaries that must be intelligently synthesized into the final output S_{MoA} . The Mixture of Agents paradigm [Wang et al., 2025] combines multiple perspectives through iterative refinement. However, existing MoA approaches apply uniform weighting to all agents, which ignores varying quality across inputs. We propose Adaptive Multi-Perspective Fusion (AMF), which extends MoA with adaptive prioritization based on quality signals and domain strengths. As shown in Figure 1, AMF processes summaries r_{ext} , r_{kg} , r_{abs} and metadata $\mathcal{M}_{\text{ext}}$, $\mathcal{M}_{\text{kg}}$, $\mathcal{M}_{\text{abs}}$ through four stages (Figure 3). This coordination mechanism distinguishes our architectural contribution from standard prompting approaches.

3.4.1 Stage 1: Perspective Quality Analysis

Stage 1 extracts quality signals from agent metadata, as shown in Figure 3. Each agent response $R_{\text{agent}} = \{r_{\text{agent}}, \mathcal{M}_{\text{agent}}\}$ already contains quality metrics computed during summary generation. We filter relevant indicators from these metadata packages without additional computation. For Extractor Agent, we extract from $\mathcal{M}_{\text{ext}}$:

$$\mathbf{q}_{\text{ext}} = \{q_{\text{salience}}, q_{\text{density}}, q_{\text{coverage}}\}, \quad (14)$$

where q_{salience} reflects sentence importance, q_{density} measures factual content, and q_{coverage} assesses entity representation.

Similarly, for KGSum Agent, we extract from $\mathcal{M}_{\text{kg}}$:

$$\mathbf{q}_{\text{kg}} = \{q_{\text{relation}}, q_{\text{contrast}}, q_{\text{entity}}\}, \quad (15)$$

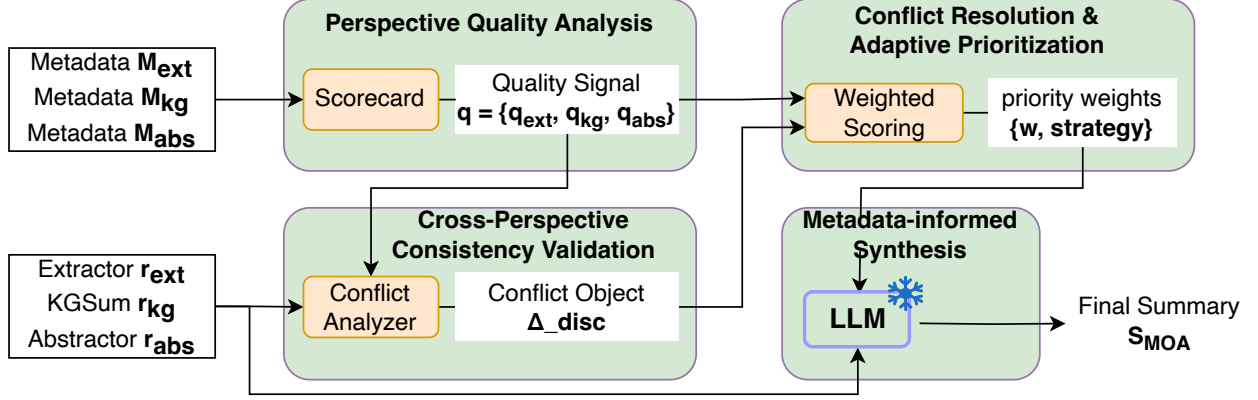


Figure 3: The four-stage AMF pipeline for adaptive multi-perspective fusion. Stage 1 analyzes quality signals from each agent’s metadata. Stage 2 validates cross-perspective consistency and detects conflicts. Stage 3 resolves conflicts and calculates adaptive priority weights. Stage 4 synthesizes the final summary using metadata-informed prompting. This architecture distinguishes our contribution from standard uniform-weighting MoA approaches.

where q_{relation} quantifies relation confidence, q_{contrast} captures contrastive flags, and q_{entity} measures coverage. For Abstractor Agent, we extract from $\mathcal{M}_{\text{abs}}$:

$$\mathbf{q}_{\text{abs}} = \{q_{\text{fluency}}, q_{\text{coherence}}, q_{\text{synthesis}}\}, \quad (16)$$

where q_{fluency} evaluates linguistic quality, $q_{\text{coherence}}$ measures narrative flow, and $q_{\text{synthesis}}$ assesses integration. These filtered signals enable Stage 2 consistency validation.

3.4.2 Stage 2: Cross-Perspective Consistency Validation

Building on the quality signals from Stage 1, Stage 2 validates consistency across summaries $\{r_{\text{ext}}, r_{\text{kg}}, r_{\text{abs}}\}$, as shown in Figure 3. We extract numerical values and named entities from each summary using regex patterns and SpaCy NER. For numerical consistency, we compute pairwise differences between extracted numbers across agents. For entity consistency, we measure overlap using Jaccard similarity on entity sets. The consistency analysis produces:

$$\Delta_{\text{disc}} = \{\text{contradictions}, \text{unique_contributions}, \text{consistency_score}\}, \quad (17)$$

where *contradictions* lists conflicting facts between agents, *unique_contributions* identifies perspective-specific insights, and *consistency_score* aggregates numerical and entity agreement. Specifically, consistency score is computed as:

$$\text{consistency_score} = \frac{1}{3} (\text{num_agreement} + \text{entity_overlap} + \text{fact_overlap}), \quad (18)$$

where each component ranges from 0 to 1. This structured output guides Stage 3 prioritization.

3.4.3 Stage 3: Conflict Resolution and Adaptive Prioritization

Stage 3 implements adaptive weighting, the core innovation distinguishing AMF from standard MoA (Figure 3). Using quality signals from Stage 1 and consistency analysis from Stage 2, we calculate agent-specific priority weights. Unlike uniform weighting in existing MoA, our approach adjusts priorities dynamically based on input characteristics. For each agent, the base quality score aggregates filtered signals:

$$\text{score}_{\text{agent}} = \text{mean}(\mathbf{q}_{\text{agent}}), \quad (19)$$

where the mean aggregates quality signal components.

Subsequently, we adjust scores with domain-specific bonuses and consistency-based penalties. For KGSum Agent, we add bonuses when relation confidence exceeds thresholds. For Extractor Agent, bonuses apply when salience scores are high. For Abstractor Agent, bonuses reward fluency. Moreover, agents with low consistency scores receive penalties. The final priority weight is:

$$w_{\text{agent}} = \frac{\text{score}_{\text{agent}} + \text{bonus}_{\text{agent}} - \text{penalty}_{\text{agent}}}{\sum_{\text{all}} (\text{score} + \text{bonus} - \text{penalty})}, \quad (20)$$

where normalization ensures $\sum_{\text{agent}} w_{\text{agent}} = 1$. These weights $\mathbf{w} = \{w_{\text{ext}}, w_{\text{kg}}, w_{\text{abs}}\}$ prioritize high-quality agents for each input.

Furthermore, we select fusion strategy based on weight distribution:

$$\text{strategy} = \begin{cases} \text{KG_LED} & \text{if } w_{\text{kg}} > 0.5 \text{ and } q_{\text{relation}} > 0.7 \\ \text{EXTRACTOR_LED} & \text{if } w_{\text{ext}} > 0.5 \text{ and } q_{\text{salience}} > 0.7 \\ \text{ABSTRACTOR_LED} & \text{if } w_{\text{abs}} > 0.5 \text{ and } q_{\text{fluency}} > 0.7 \\ \text{ADAPTIVE} & \text{otherwise} \end{cases}, \quad (21)$$

where thresholds determine when one agent dominates. This strategy guides Stage 4 synthesis.

3.4.4 Stage 4: Metadata-Informed Synthesis

Stage 4 synthesizes the final summary using metadata from Stages 1 to 3 (Figure 3). We consolidate all analysis components into:

$$\mathcal{A}_{\text{analysis}} = \{\mathbf{q}, \Delta_{\text{disc}}, \mathbf{w}, \text{strategy}\}, \quad (22)$$

where quality signals, discrepancies, weights, and strategy guide synthesis.

The fusion prompt Π_{AMF} embeds this metadata to direct LLM attention (Figure 4). Unlike standard prompting with equal attention, our prompt explicitly specifies priority weights and conflict resolution. The final summary is:

$$S_{\text{MoA}} = \text{LLM}(\Pi_{\text{AMF}}, \{r_{\text{ext}}, r_{\text{kg}}, r_{\text{abs}}\}, \mathcal{A}_{\text{analysis}}), \quad (23)$$

where the LLM receives summaries with explicit guidance from adaptive prioritization. This architectural coordination mechanism demonstrates that performance gains stem from intelligent multi-agent orchestration rather than simple prompting. AMF produces comprehensive metadata $\mathcal{M}_{\text{AMF}}$ for transparency.

4 Experiments

In this section, we present a comprehensive experimental evaluation of our proposed MoA framework for MDS. This evaluation is conducted on diverse real-world datasets. MoA is designed as a training-free, multi-agent architecture, while both SOTA full-training (supervised) and training-free (unsupervised) baselines are considered. Through rigorous comparisons with SOTA baselines and in-depth analyses, we aim to address the following research questions:

- **RQ1:** How does the choice of LLM (open-source vs. closed-source) influence MoA’s performance? Can MoA effectively leverage more advanced LLMs?
- **RQ2:** How does MoA perform on MDS tasks compared to SOTA supervised and unsupervised baselines?
- **RQ3:** What is the contribution of each agent within the MoA framework to the overall summarization quality?

We first investigate the impact of different LLM choices (both closed-source and open-source) on MoA’s efficacy to answer RQ1. Based on these findings, we select the optimal LLM. We then establish MoA’s performance against SOTA baselines (RQ2) over both English and Vietnamese datasets. Finally, an ablation study is carried out to dissect individual component contributions to answer RQ3.

4.1 Experimental Setup

Evaluation Datasets We evaluate MoA on four diverse MDS datasets spanning multiple domains and languages: Multi-News (news articles, English) [Fabbri et al., 2019], Multi-XScience (scientific papers, English) [Lu et al., 2020], VN-MDS (news articles, Vietnamese)², and ViMs (diverse documents, Vietnamese) [Tran et al., 2020]. Table 1 presents comprehensive statistics for all datasets, including test sample counts, number of reference summaries per cluster, average document lengths, and summary lengths. Our evaluation encompasses substantial scale variation. Document quantities range from 29 samples (VN-MDS) to 5,622 samples (Multi-News), while average document lengths span from 419 words to 2,103 words. The number of reference summaries per cluster varies from 2.79 to 6.56, reflecting diverse multi-document aggregation

²<https://github.com/lupanh/VietnameseMDS>

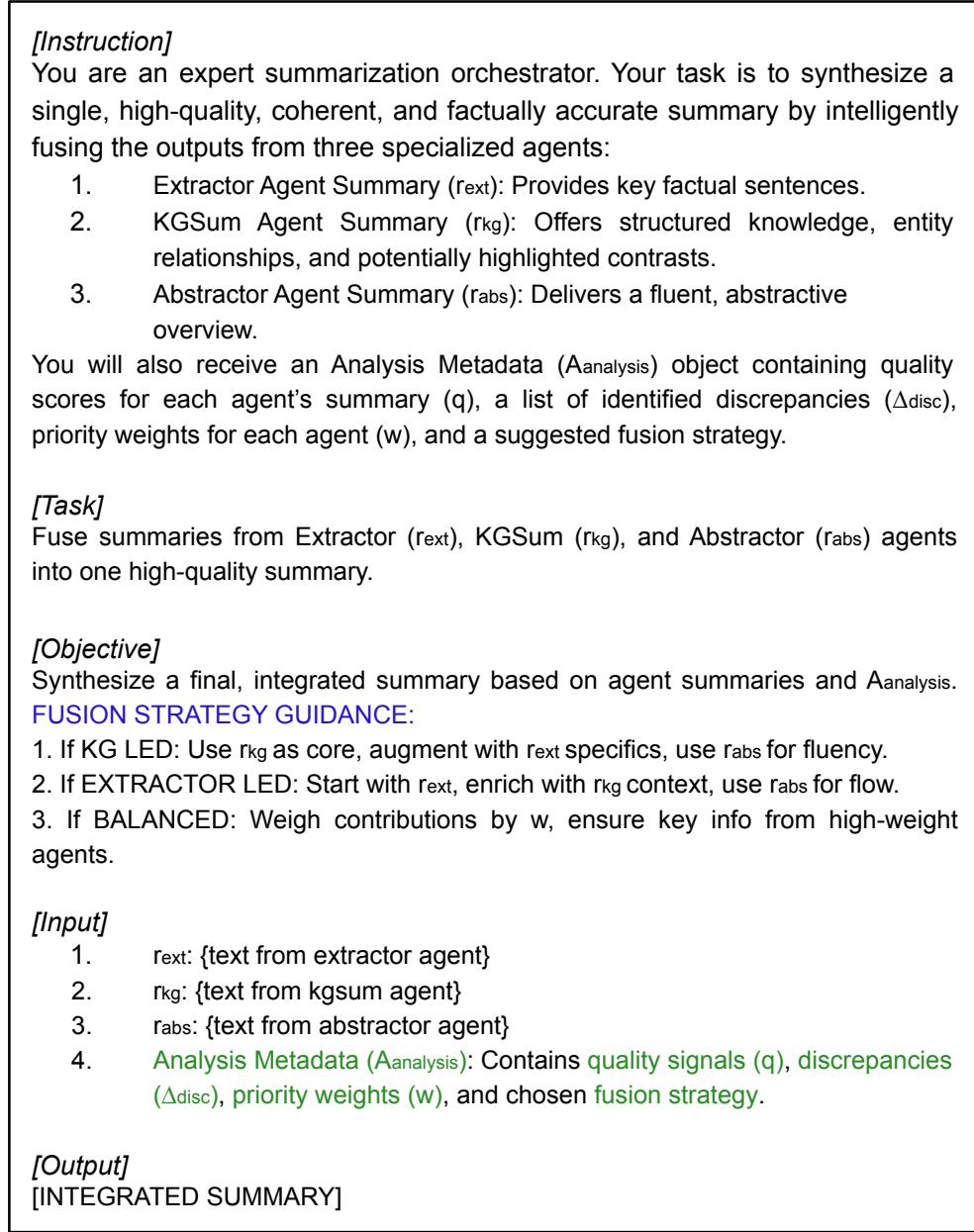


Figure 4: Structure of the enhanced fusion prompt Π_{AMF} used in Stage 4 of AMF. The prompt dynamically incorporates quality signals q , priority weights w , discrepancy information Δ_{disc} , and the selected fusion strategy. This metadata-driven prompting enables the LLM to perform intelligent synthesis by explicitly prioritizing high-quality perspectives and resolving conflicts based on quantitative analysis.

requirements. Therefore, our evaluation validates architectural effectiveness across realistic scale variations encountered in established MDS benchmarks. (Detailed descriptions of dataset characteristics, preprocessing procedures, and complexity analysis can be found in Appendix C.)

Evaluation Metrics Following previous works [Zhang et al., 2020], we use ROUGE F1-scores as standard evaluation metrics: ROUGE-1 (R1) for content overlap, ROUGE-2 (R2) for fluency and local coherence, and ROUGE-L (RL) for structural similarity and overall coherence against reference summaries [Lin, 2004]. Higher scores indicate better summarization quality.

Table 1: Statistics of evaluation datasets

Dataset	Language	Test Samples	# refs	doc. len	summ. len
Multi-News	English	5,622	2.79	2,103.49	263.66
Multi-XScience	English	5,093	4.42	778.08	116.44
VN-MDS	Vietnamese	29	3.14	419.0	172.1
ViMs	Vietnamese	45	6.56	470.2	231.2

Reporting Conventions All ROUGE scores are reported as percentages. In RQ2, MoA scores are reported as mean $\pm$ standard deviation over three runs, while baseline scores are taken from the original publications. RQ1 and RQ3 results are shown as mean scores only. Bold entries indicate the best score in a column, and underlined entries indicate the second-best. A hyphen “-” denotes that a method was not evaluated on that dataset or the original paper did not report that metric. Baseline scores are taken from the original publications. Abbreviations used in tables: FT = Full Training, TF = Training-Free, Prop. = Proprietary, Open = Open-source.

Implementation Details MoA is a training-free framework that can use different backbone LLMs, although its quality still depends on backbone suitability. We systematically evaluate MoA with four leading closed-source LLMs (GPT-4.1-mini³, Grok-3-mini⁴, GPT-4o-mini⁵, Gemini-1.5-flash-8b⁶) and three representative open-source LLMs (Llama-3.1-8B-Instruct⁷, Qwen2.5-7B-Instruct⁸, Gemma-2-9b-it⁹). All experiments are conducted on NVIDIA Quadro RTX 8000 GPU. We perform three independent runs for statistical validation, with mean and standard deviation reported. (Complete hyperparameter configurations and detailed parameter selection procedures are provided in Appendix D. API cost analysis for closed-source models is provided in Appendix E.) For ablation studies, we utilize the few-shot prompt template Π_{few} presented in Appendix A for single LLM baseline comparisons.

Baseline Selection We compare MoA against two types of baselines. Full-training (supervised) baselines include PEGASUS [Zhang et al., 2020], LED [Beltagy et al., 2020], PRIMERA [Xiao et al., 2022], and Centrum [Puduppully et al., 2023] for English, and LatVisNewshead [Le and Le, 2025] for Vietnamese. Training-free baselines include SRI+beam [Zhang et al., 2023] and GLIMMER-TTR [Liu et al., 2024] for English, and Graph combine Abstractive [Doan-Thanh et al., 2023] and Bert-VBD [Vuong et al., 2024] for Vietnamese. Training-free methods use pre-trained LLMs without additional fine-tuning on MDS datasets. Therefore, they do not require task-specific data collection or labeling, which is useful when specialized training data is limited or computational resources are constrained.

4.2 Impact of LLM Choice on MoA Performance (RQ1)

Table 2: Performance comparison of our proposed MoA with different LLM backbones on English datasets

Method	Type	Multi-News			Multi-XScience		
		R1	R2	RL	R1	R2	RL
GPT-4.1-mini	Prop.	54.97	<u>24.90</u>	29.40	36.66	5.64	19.73
Grok-3-mini	Prop.	<u>53.91</u>	24.68	28.61	<u>35.60</u>	<u>5.42</u>	<u>18.94</u>
GPT-4o-mini	Prop.	52.67	23.72	27.97	31.05	4.72	16.28
Gemini-1.5-flash-8b	Prop.	51.92	23.10	27.51	30.11	4.50	15.82
Llama-3.1-8b-Instruct	Open	52.40	25.98	<u>28.90</u>	23.42	2.60	13.24
Qwen2.5-7B-Instruct	Open	48.17	21.75	26.55	23.64	2.55	12.63
Gemma-2-9b-it	Open	25.08	8.48	14.81	24.05	2.47	12.84

³<https://openai.com/index/gpt-4-1/>

⁴<https://x.ai/news/grok-3>

⁵<https://openai.com/index/hello-gpt-4o/>

⁶<https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/>

⁷<https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>

⁸<https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>

⁹<https://huggingface.co/google/gemma-2-9b-it>

Table 3: Performance comparison of our proposed MoA with different LLM backbones on Vietnamese datasets

Method	Type	VN-MDS			ViMs		
		R1	R2	RL	R1	R2	RL
GPT-4.1-mini	Prop.	80.11	52.32	48.29	80.26	55.57	49.04
Grok-3-mini	Prop.	<u>79.05</u>	<u>52.10</u>	<u>47.50</u>	<u>79.20</u>	<u>55.35</u>	<u>48.25</u>
GPT-4o-mini	Prop.	72.90	46.10	40.50	68.80	49.50	39.00
Gemini-1.5-flash-8b	Prop.	72.25	45.60	40.02	68.05	48.80	38.42
Llama-3.1-8b-Instruct	Open	71.80	45.20	39.80	67.50	48.50	38.50
Qwen2.5-7B-Instruct	Open	65.10	40.50	38.80	61.00	41.00	34.00
Gemma-2-9b-it	Open	63.20	38.70	37.10	59.00	39.00	32.00

Tables 2 and 3 present comprehensive performance comparison of MoA with different LLM backbones across all four evaluation datasets. Since MoA operates as a multi-agent system where LLMs serve as the primary reasoning engines for each agent, we systematically investigate how different LLM capabilities affect overall architectural performance. The results reveal clear performance hierarchies and important insights about LLM selection for MDS tasks.

On English datasets (Table 2), closed-source models demonstrate substantial superiority over open-source alternatives. Specifically, GPT-4.1-mini achieves the highest scores across all metrics: 54.97 R1, 24.90 R2, and 29.40 RL on Multi-News, along with 36.66 R1, 5.64 R2, and 19.73 RL on Multi-XScience. These scores surpass Grok-3-mini by 2.0 percent in R1 on Multi-News and 3.0 percent on Multi-XScience, demonstrating consistent leadership. Moreover, GPT-4o-mini and Gemini-1.5-flash-8b achieve competitive results with 52.67 R1 and 51.92 R1 on Multi-News respectively, validating their strong general capabilities. In particular, among open-source LLMs, Llama-3.1-8b-Instruct achieves the best performance with 52.40 R1 on Multi-News, substantially outperforming other open-source alternatives. However, Gemma-2-9b-it, designed as a multimodal model, shows suboptimal performance with only 25.08 R1 on Multi-News, indicating that specialized capabilities do not always transfer effectively to text-only MDS applications.

On Vietnamese datasets (Table 3), the performance hierarchy remains consistent with English results. GPT-4.1-mini achieves 80.11 R1 on VN-MDS and 80.26 R1 on ViMs, closely followed by Grok-3-mini with 79.05 R1 and 79.20 R1 respectively. The minimal performance gap (1.3 percent on VN-MDS, 1.3 percent on ViMs) confirms both models’ strong multilingual capabilities. Furthermore, the performance differences between Vietnamese and English datasets primarily reflect dataset complexity characteristics rather than language-specific advantages. Specifically, Vietnamese datasets exhibit significantly higher extractiveness (94.3 percent versus 19.2 percent for English) and content redundancy, inflating ROUGE scores through exact n-gram matching rather than demanding abstractive synthesis. Detailed comparative complexity analysis with supporting metrics is presented in Table 12 of Appendix C.2.

The results show that MoA performs best with strong closed-source models, while suitable instruction-tuned open-source models such as Llama-3.1-8B-Instruct and Qwen2.5-7B-Instruct still achieve competitive performance. By contrast, when MoA uses a less suitable backbone such as Gemma-2-9b-it, the overall summarization quality decreases noticeably. Therefore, GPT-4.1-mini is selected for the main comparative evaluation in RQ2 and the controlled ablation study in RQ3.

4.3 Comparative Performance Evaluation (RQ2)

To address RQ2, we evaluate MoA’s performance against established full training and training-free baselines. For these comparisons, MoA utilizes GPT-4.1-mini as its backbone LLM based on its superior overall performance demonstrated in RQ1 (Section 4.2). Furthermore, we conduct three independent experimental runs to ensure result reliability, with mean and standard deviation reported for our method. Moreover, due to computational resource constraints and API cost considerations for large-scale evaluation across multiple datasets, we perform three runs as a practical balance between statistical validation and feasibility. Comprehensive results are presented in Tables 4 and 5 for English and Vietnamese datasets, respectively.

Tables 4 and 5 present comprehensive performance comparison across all four evaluation datasets. The results demonstrate MoA’s consistent superiority over existing methods across diverse evaluation settings.

Table 4: Performance comparison on English summarization datasets

Method	Type	Multi-News			Multi-XScience		
		R1	R2	RL	R1	R2	RL
PEGASUS	FT	32.0	10.0	17.0	28.0	<u>5.0</u>	15.0
LED	FT	17.0	4.0	10.0	15.0	<u>2.0</u>	10.0
PRIMERA	FT	42.0	14.0	21.0	29.0	<u>5.0</u>	16.0
Centrum	FT	<u>46.0</u>	<u>17.0</u>	<u>23.0</u>	-	-	-
SRI+beam	TF	44.0	15.0	19.0	-	-	-
GLIMMER-TTR	TF	44.0	15.0	21.0	<u>32.0</u>	<u>5.0</u>	<u>17.0</u>
MoA (Ours)	TF	55.0±0.3	24.9±0.4	29.4±0.3	36.7±0.2	5.6±0.1	19.7±0.2

Table 5: Performance comparison on Vietnamese summarization datasets

Method	Type	VN-MDS			ViMs		
		R1	R2	RL	R1	R2	RL
Graph combine Abs	TF	69.0	35.0	33.0	-	-	-
Bert-VBD	TF	70.0	40.0	34.0	-	-	-
LatVisNewshead	FT	<u>77.0</u>	<u>50.0</u>	<u>47.0</u>	<u>79.0</u>	<u>55.0</u>	<u>48.0</u>
MoA (Ours)	TF	80.1±0.5	52.3±0.7	48.3±0.6	80.3±0.5	55.6±0.8	49.0±0.6

On English datasets, our framework establishes new state-of-the-art results as shown in Table 4. On Multi-News, MoA achieves 55.0±0.3 R1, 24.9±0.4 R2, and 29.4±0.3 RL. These scores substantially outperform both supervised and training-free baselines. Specifically, MoA improves over Centrum by 19.6 percent (from 46.0 to 55.0) in R1 and surpasses GLIMMER-TTR by 25.0 percent in R1. On Multi-XScience, MoA achieves 36.7±0.2 R1, surpassing GLIMMER-TTR by 14.7 percent. Moreover, MoA demonstrates substantial improvements in R2 and RL metrics: 46.5 percent improvement over Centrum in R2 (from 17.0 to 24.9) and 27.8 percent in RL (from 23.0 to 29.4) on Multi-News. This validates robust generalization to specialized scientific domains where precise terminology and complex relationships require sophisticated coordination.

The framework demonstrates language-agnostic effectiveness on Vietnamese datasets as presented in Table 5. On VN-MDS, MoA achieves 80.1±0.5 R1, 52.3±0.7 R2, and 48.3±0.6 RL. On ViMs, MoA reaches 80.3±0.5 R1, 55.6±0.8 R2, and 49.0±0.6 RL. In particular, MoA surpasses the specialized Vietnamese model LatVisNewshead by 4.0 percent in R1 on VN-MDS (from 77.0 to 80.1) and 1.6 percent on ViMs (from 79.0 to 80.3) despite operating in training-free mode. Furthermore, MoA achieves consistent improvements across all metrics: 4.6 percent in R2 and 2.8 percent in RL on VN-MDS, and 1.1 percent in R2 and 2.1 percent in RL on ViMs. This validates that our architectural coordination approach successfully leverages multilingual LLM capabilities without language-specific fine-tuning. Tables 2 and 3 further show that MoA remains competitive with several reported baselines when it uses suitable open-source backbones such as Llama-3.1-8B-Instruct and Qwen2.5-7B-Instruct. In contrast, the results of Gemma-2-9b-it show that MoA becomes less competitive when the backbone is less suitable for this task.

Consistent improvements across diverse document types further validate architectural robustness. The framework handles news articles (Multi-News, VN-MDS), scientific papers (Multi-XScience), and diverse documents (ViMs) with systematic performance gains ranging from 1.6 percent to 46.5 percent across different metrics and datasets. Moreover, statistical validation presented in Appendix F confirms that MoA significantly outperforms all baselines across 12 dataset-metric combinations with $p < 0.001$. Therefore, the multi-agent architecture with knowledge graph reasoning and adaptive fusion provides generalizable solutions for complex inter-document relationship modeling.

4.4 Ablation Study (RQ3)

Table 6 presents comprehensive ablation study results investigating the individual contribution of each agent within the MoA framework to address RQ3. We systematically evaluate single-agent performance, multi-agent combinations, and the full framework using GPT-4.1-mini on Multi-News and VN-MDS datasets. Here,

Table 6: Ablation study results for our proposed MoA using GPT-4.1-mini

Configuration	Multi-News (English)			VN-MDS (Vietnamese)		
	R1	R2	RL	R1	R2	RL
<i>Part 1: Single-Agent Performance and LLM Baseline</i>						
LLM with structured prompt	27.0	5.0	11.0	39.3	6.6	15.2
Abstractor agent only	31.0	3.0	12.0	45.2	3.9	16.6
Extractor agent only	33.0	6.0	17.0	48.1	7.9	23.5
KGSum agent only	37.0	4.0	12.0	59.8	9.2	26.2
<i>Part 2: Multi-Agent Combinations and Full Framework</i>						
MoA w/o KGSum agent	48.9	21.1	25.9	70.0	44.7	44.3
MoA w/o Abstractor agent	<u>51.5</u>	<u>23.2</u>	<u>27.3</u>	<u>73.8</u>	<u>48.2</u>	<u>45.8</u>
MoA w/o Extractor agent	44.8	18.7	23.3	65.1	38.1	41.9
Full MoA framework	55.0	24.9	29.4	80.1	52.3	48.3

the backbone LLM is intentionally fixed so that RQ3 can focus on the contribution of KGSum, Abstractor, and Extractor within one consistent MoA setting.

KGSum agent demonstrates strongest single-agent performance. Among individual agents, KGSum achieves 37.0 R1 on Multi-News and 59.8 R1 on VN-MDS, substantially outperforming Extractor (33.0 R1, 48.1 R1) and Abstractor (31.0 R1, 45.2 R1). Moreover, KGSum surpasses the simple LLM baseline (27.0 R1, 39.3 R1) by 37.0 percent and 52.2 percent respectively. This validates the effectiveness of knowledge graph construction and community-based reasoning for capturing inter-document relationships, confirming KGSum as the core architectural component for MDS tasks.

Multi-agent combinations consistently outperform single agents. All two-agent configurations achieve superior performance compared to individual agents, demonstrating complementary agent interactions. Specifically, MoA without Abstractor reaches 51.5 R1 on Multi-News, while MoA without KGSum achieves 48.9 R1. In particular, removing KGSum produces the worst degradation: MoA without KGSum scores only 48.9 R1 versus 51.5 R1 for MoA without Abstractor, an 11.2 percent performance gap. Furthermore, MoA without Extractor achieves 44.8 R1, demonstrating that factual content extraction remains essential for coordination. These patterns validate that KGSum provides critical structural understanding that other agents cannot compensate for, emphasizing its central role in multi-agent coordination.

Full MoA framework with AMF achieves optimal performance. The complete three-agent MoA reaches 55.0 R1 on Multi-News and 80.1 R1 on VN-MDS, outperforming all ablated configurations. The full framework improves over the best two-agent combination (MoA without Abstractor: 51.5 R1, 73.8 R1) by 6.8 percent and 8.5 percent respectively. Therefore, the AMF mechanism effectively integrates complementary perspectives from all three agents, while the complete MoA architecture provides a generalizable solution for complex multi-document summarization through coordinated knowledge graph reasoning and adaptive multi-agent fusion.

4.5 Human Evaluation

To verify the practical quality of summaries generated by our MoA framework, we conduct human evaluation beyond automatic metrics. Automatic metrics like ROUGE measure lexical overlap but cannot fully capture summary fluency and coherence. Therefore, systematic human judgment provides essential validation of real-world summary quality.

Following established evaluation guidelines from DUC 2007¹⁰, we assess summaries across four key dimensions: grammaticality (linguistic correctness and sentence structure quality), non-redundancy (absence of repetitive information), referential clarity (pronoun and entity reference understandability), and structure coherence (logical organization and information flow). These four dimensions provide comprehensive coverage of summary quality from human perspective.

For experimental settings, we randomly select 20 document clusters from Multi-News for English and 20 clusters from VN-MDS for Vietnamese. We recruit 100 participants to evaluate generated summaries. Each participant rates summaries on a scale from 1 to 5 for each dimension, where higher scores indicate better

¹⁰<https://www-nlpir.nist.gov/projects/duc/duc2007/quality-questions.txt>

quality. The annotation interface and detailed evaluation criteria are presented in Appendix G. This setup balances evaluation coverage with practical feasibility for large-scale assessment. Table 7 presents human evaluation results on Multi-News comparing MoA against two strong baselines (Centrum and PRIMERA), while Table 8 shows results on VN-MDS comparing MoA against BertVBD.

Table 7: Human evaluation results on Multi-News dataset (English)

Model	Gram.	Non-red.	Ref. Clar.	Str. & Coh.
Centrum	3.57	3.35	3.67	3.37
PRIMERA	2.75	3.15	3.10	2.75
MoA (Ours)	4.56	4.67	4.55	4.47

Table 7 presents human evaluation results on Multi-News dataset. We compare MoA against two strong baselines: Centrum (supervised full-training) and PRIMERA (supervised). MoA achieves consistent superiority across all four evaluation dimensions.

MoA demonstrates strong performance across all quality dimensions. Specifically, MoA achieves 4.56 in grammaticality, 4.67 in non-redundancy, 4.55 in referential clarity, and 4.47 in structure coherence. These scores substantially outperform both baselines. In particular, MoA improves non-redundancy by 39.4 percent over Centrum (from 3.35 to 4.67). Moreover, MoA achieves 27.7 percent improvement in grammaticality over Centrum (from 3.57 to 4.56). Furthermore, referential clarity improves by 24.0 percent (from 3.67 to 4.55), while structure coherence gains 32.6 percent (from 3.37 to 4.47).

Baseline performance reveals important quality gaps. PRIMERA demonstrates the weakest results, scoring only 2.75 in grammaticality and structure coherence. This indicates significant limitations in language generation quality for supervised approaches. Therefore, MoA’s multi-agent coordination produces summaries with superior fluency and logical organization compared to traditional supervised methods.

Table 8: Human evaluation results on VN-MDS dataset (Vietnamese)

Model	Gram.	Non-red.	Ref. Clar.	Str. & Coh.
MoA (Ours)	4.20	4.34	4.29	4.09
BertVBD	3.93	4.20	4.02	3.69

Table 8 presents human evaluation results on VN-MDS dataset. We compare MoA against BertVBD, a competitive training-free Vietnamese baseline. MoA achieves consistent superiority across all evaluation dimensions, validating language-agnostic effectiveness.

MoA demonstrates strong performance on Vietnamese summarization tasks. Specifically, MoA achieves 4.20 in grammaticality, 4.34 in non-redundancy, 4.29 in referential clarity, and 4.09 in structure coherence. These scores outperform BertVBD across all dimensions. In particular, structure coherence improves by 10.8 percent (from 3.69 to 4.09). Moreover, grammaticality gains 6.9 percent (from 3.93 to 4.20), while referential clarity increases by 6.7 percent (from 4.02 to 4.29). Furthermore, non-redundancy improves by 3.3 percent (from 4.20 to 4.34).

Cross-language validation confirms architectural robustness. The consistent pattern of MoA superiority across both English and Vietnamese datasets demonstrates language-agnostic effectiveness. MoA maintains strong performance (4.09 to 4.67 range) across diverse linguistic contexts without language-specific tuning. Therefore, human evaluation results complement automatic metrics, providing convergent evidence that MoA produces summaries with superior fluency, coherence, and overall quality from human perspective.

5 Conclusions and Future Work

We present MoA, a novel architectural framework for training-free multi-document summarization that coordinates three specialized agents. Our approach delivers three key innovations that advance the field. First, the KGSum agent constructs knowledge graphs to model inter-document relationships and contradictions explicitly. Second, the AMF mechanism fuses agent outputs using metadata-driven strategies. Third, the framework operates without task-specific training while maintaining systematic coordination for complex multi-document scenarios. Furthermore, experimental validation across diverse datasets demonstrates consistent performance gains over both training-free and supervised baselines, establishing the effectiveness of

our architectural approach in learning complex inter-document relationships. Moreover, human evaluation confirms superior summary quality improvements in coherence, informativeness, and conciseness.

Our framework successfully leverages existing language models while providing architectural innovations that enhance their capability to handle complex multi-document relationships. Furthermore, evaluation on established benchmarks provides solid validation for our architectural approach. Future research will extend this framework to specialized domains including technical, legal, and creative content where complex inter-document relationships present greater challenges. We will also examine how different backbone LLMs influence the relative contribution of each agent within multi-agent summarization frameworks, including our proposed MoA framework.

References

- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.
- Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better llm-based evaluators through multi-agent debate. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Representation Learning*, volume 2024, pages 9079–9093, 2024.
- Tam Doan-Thanh, Cam-Van Thi Nguyen, Huu-Thin Nguyen, Mai-Vu Tran, and Quang Thuy Ha. Vietnamese multidocument summarization using subgraph selection-based approach with graph-informed self-attention mechanism. In Ngoc Thanh Nguyen, Siridech Boonsang, Hamido Fujita, Bogumiła Hnatkowska, Tzung-Pei Hong, Kitsuchart Pasupa, and Ali Selamat, editors, *Intelligent Information and Database Systems*, pages 259–271, Singapore, 2023. Springer Nature Singapore. ISBN 978-981-99-5837-5.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Günes Erkan and Dragomir R Radev. Lexrank: Graph-based lexical centrality as salience in text summarization. *Journal of artificial intelligence research*, 22:457–479, 2004.
- Alexander Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir Radev. Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1074–1084, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1102.
- Jade Goldstein and Jaime Carbonell. Summarization: (1) using MMR for diversity- based reranking and (2) evaluating summaries. In *TIPSTER TEXT PROGRAM PHASE III: Proceedings of a Workshop held at Baltimore, Maryland, October 13-15, 1998*, pages 181–195, Baltimore, Maryland, USA, October 1998. Association for Computational Linguistics. doi: 10.3115/1119089.1119120.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. Knowledge graphs. *ACM Computing Surveys (Csur)*, 54(4):1–37, 2021.
- The Anh Le and Hai Son Le. Latvis: Large-scale task-specific language model for low-resource vietnamese multi-document summarization. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 2025.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Bohan Li, Hao Zhou, Junxian He, Mingxuan Wang, Yiming Yang, and Lei Li. On the sentence embeddings from pre-trained language models. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors,

- Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9119–9130, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.733.
- Miao Li, Jianzhong Qi, and Jey Han Lau. Compressed heterogeneous graph for abstractive multi-document summarization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 13085–13093, 2023.
- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.
- Ran Liu, Ming Liu, Min Yu, Jianguo Jiang, Gang Li, Dan Zhang, Jingyuan Li, Xiang Meng, and Weiqing Huang. Glimmer: Incorporating graph and lexical features in unsupervised multi-document summarization. *arXiv preprint arXiv:2408.10115*, 2024.
- Yao Lu, Yue Dong, and Laurent Charlin. Multi-xscience: A large-scale dataset for extreme multi-document summarization of scientific articles. *arXiv preprint arXiv:2010.14235*, 2020.
- Congbo Ma, Wei Emma Zhang, Mingyu Guo, Hu Wang, and Quan Z Sheng. Multi-document summarization via deep learning techniques: A survey. *ACM Computing Surveys*, 55(5):1–37, 2022.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- Rada Mihalcea and Paul Tarau. TextRank: Bringing order into texts. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 404–411, Barcelona, Spain, 2004. Association for Computational Linguistics.
- Muhammad Firoz Mridha, Aklima Akter Lima, Kamruddin Nur, Sujoy Chandra Das, Mahmud Hasan, and Muhammad Mohsin Kabir. A survey of automatic text summarization: Progress, process and challenges. *IEEE Access*, 9:156043–156070, 2021.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- Ratish Surendran Puduppully, Parag Jain, Nancy Chen, and Mark Steedman. Multi-document summarization with centroid-based pretraining. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 128–138, Toronto, Canada, July 2023. Association for Computational Linguistics.
- Dragomir Radev, Timothy Allison, Matthew Craig, Stanko Dimitrov, Kareem Omer, Michael Topper, Adam Winkel, and Jin Yi. A scalable multi-document centroid-based summarizer. In *Demonstration Papers at HLT-NAACL 2004*, pages 23–25, Boston, Massachusetts, USA, May 2 - May 7 2004. Association for Computational Linguistics.
- Gaetano Rossiello, Pierpaolo Basile, and Giovanni Semeraro. Centroid-based text summarization through compositionality of word embeddings. pages 12–21, Valencia, Spain, April 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-1003.
- Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1099.
- Hassan Shakil, Ahmad Farooq, and Jugal Kalita. Abstractive text summarization: State of the art, challenges, and improvements. *Neurocomputing*, page 128255, 2024.
- Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.
- Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1):1–12, 2019.
- Nhi-Thao Tran, Minh-Quoc Nghiem, Nhung TH Nguyen, Ngan Luu-Thuy Nguyen, Nam Van Chi, and Dien Dinh. Vims: a high-quality vietnamese dataset for abstractive multi-document summarization. *Language Resources and Evaluation*, 54(4):893–920, 2020.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

- Tuan-Cuong Vuong, Trang Mai Xuan, and Thien Van Luong. Bert-vbd: Vietnamese multi-document summarization framework. *arXiv preprint arXiv:2409.12134*, 2024.
- Junlin Wang, Jue WANG, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- Shuai Wang, Xiang Zhao, Bo Li, Bin Ge, and Daquan Tang. Integrating extractive and abstractive models for long text summarization. In *2017 IEEE International Congress on Big Data (BigData Congress)*, pages 305–312, 2017. doi: 10.1109/BigDataCongress.2017.46.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Lidia Sam Chao, and Derek Fai Wong. A survey on llm-generated text detection: Necessity, methods, and future directions. *Computational Linguistics*, pages 1–66, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Wen Xiao, Iz Beltagy, Giuseppe Carenini, and Arman Cohan. PRIMERA: Pyramid-based masked sentence pre-training for multi-document summarization. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5245–5263, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.360.
- Haopeng Zhang, Sangwoo Cho, Kaiqiang Song, Xiaoyang Wang, Hongwei Wang, Jiawei Zhang, and Dong Yu. Unsupervised multi-document summarization with holistic inference. *arXiv preprint arXiv:2309.04087*, 2023.
- Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter Liu. Pegasus: Pre-training with extracted gap-sentences for abstractive summarization. In *International conference on machine learning*, pages 11328–11339. PMLR, 2020.
- Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B Hashimoto. Benchmarking large language models for news summarization. *Transactions of the Association for Computational Linguistics*, 12:39–57, 2024.

Appendix A Prompt Templates for MoA Framework

This appendix presents all prompt templates used throughout the MoA framework. These include KGSum Agent prompts for entity extraction (Π_e), relation classification (Π_{rel}), community summarization (Π_s), and meta-aware fusion (Π_f), along with Abstractor Agent prompts for initial generation (Π_{init}) and iterative refinement (Π_{ref}), and baseline few-shot prompts (Π_{few}) for comparison.

```
[Instruction]
You are a multilingual expert entity-relationship extractor with advanced knowledge of semantic parsing.

[Objective]
Given:
1. A source text.
2. A closed set of entities.

Your task is to identify pairs of entities that have a meaningful, directly supported relationship in the text.
A "meaningful relationship" means:
- The text explicitly states or clearly supports that EntityA and EntityB are connected in some way (e.g. acquisition, ownership, authorship, employment, partnership, membership, leadership, location, creation, etc.).
- The relationship is described in the same sentence, OR in immediately adjacent sentences that refer to the same subject.
- The connection must come from the source text itself. Do NOT use external knowledge.
You are NOT asked to describe the relationship. You only need to output which entity pairs are related.

[CHAIN OF THOUGHT]
1. Scan & Locate
  - Read the full text and the provided entity list.
  - For each ordered pair (EntityA, EntityB) drawn from that list, find sentences where both are mentioned, OR where one is described in terms of the other.
2. Validate Relationship
  - Keep the pair ONLY if the text indicates a concrete relationship between them (e.g. "A acquired B", "A is CEO of B", "A founded B").
  - Ignore pairs that are only co-mentioned without any stated link.
  - Ignore speculative, hypothetical, or inferred links not directly supported by the text.
3. Record Pair
  - For each valid relationship, create a string with the format: "EntityA | EntityB"
  - EntityA and EntityB MUST both come exactly (verbatim) from the provided entity list. Do NOT rewrite, merge, normalize, or invent entities.

[Few-shot example]
Text evidence: "Meta acquired WhatsApp in 2014 for $19 billion."
Entities: ["Meta", "WhatsApp", "Google"]

Valid output:
[
  "Meta | WhatsApp"
]

Explanation for you (not to be returned in output): "Meta" and "WhatsApp" are linked by an explicit acquisition. "Google" is mentioned in the entity list but not in a described relationship, so "Google | WhatsApp" is not output.

[Input]
Text: {text}
Entities: {entities}

[Output]
Return ONLY a JSON array of strings.
Each string must be exactly "Entity1 | Entity2" with a single pipe and single space on each side of the pipe.

Example output:
[
  "Meta | WhatsApp",
  "WhatsApp | Meta"
]
```

Figure 5: Entity Extraction Prompt (Π_e) used by the KGSum Agent to identify relationships between entities.

[Instruction]
 You are a multilingual expert in entity-level semantic analysis. You specialize in identifying semantic relationships, including contrastive / conflicting claims, between entities mentioned in a text.

[Objective]
 Given:
 1. A source text.
 2. A closed set of entities.

Your task is to extract all meaningful relationships between pairs of entities in the provided entity list. For each relationship, you must:

- Classify the relationship using one of the allowed types.
- Assign a confidence score between 0.0 and 1.0 that reflects how directly and unambiguously the text supports this relationship.

A relationship is valid only if it is explicitly stated or clearly supported by the text. Do NOT use external knowledge or inference beyond what the text implies.

[Contrastive relation]
 - "contrastive relation" (A and B make differing / conflicting / contrasting claims, positions, measurements, interpretations, or conclusions in the text)

[CHAIN OF THOUGHT]

1. Candidate Pair Generation: Consider all ordered pairs (EntityA, EntityB) drawn from the provided entity list.
2. Relation Classification: Use "contrastive relation" ONLY if the text clearly expresses disagreement, conflicting numbers, or opposing claims between the two entities.
3. Confidence Scoring: Assign a confidence between 0.0 and 1.0.
 - Higher confidence (≥ 0.8) means the relationship is explicit and unambiguous.
 - Medium confidence (~ 0.5 – 0.79) means the relationship is implied but not directly stated word-for-word.
 - Low confidence (< 0.5) should generally not be emitted unless it still clearly aligns with the text.
4. Output Assembly
 - For each valid relationship, create an object with:
 - head, relation, tail, confidence, evidence.
 - Use entity surface forms EXACTLY as they appear in the provided entity list. Do NOT rename or normalize entities.
 - Exclude duplicate (head, relation, tail) triples.

[Input]
 Text: {text}
 Entities: {entities}

[Output]
 Return ONLY a single valid JSON array.
 Each element of the array MUST be an object with the following keys, in this exact shape:

```
[
  {
    "head": "EntityA",
    "relation": "one of: works for | part of | located in | causes | related to | contrastive relation",
    "tail": "EntityB",
    "confidence": 0.0-1.0 (number),
    "evidence": "Short textual justification grounded in the input text"
  },
  ...
]
```

If there are no valid relationships, return: []

Figure 6: Relation and Contrast Extraction Prompt (Π_{rel}) used by the KGSum Agent to classify relationships between entity pairs, including contrastive relations.

[Instruction] You are a multilingual, fact-focused summarization agent. You generate compact, objective summaries of knowledge graph communities. [Objective] Given: 1. The linearized textual representation of a single knowledge graph community (entities, facts, relationships, numbers). Your task is to produce a concise summary (strictly 1–2 sentences) that: - captures the main theme of the community, - mentions the most important entities, facts, numbers, and relationships, - stays neutral and factual. Constraints: - Do NOT include opinions, speculation, or vague language. - Do NOT generalize beyond what is in the input. - Do NOT exceed two sentences under any circumstance. [CHAIN OF THOUGHT] 1. Identify Core Focus - Determine what the community is about (e.g. a company, event, investigation, collaboration, dispute). - Note the central entities and how they are connected. 2. Extract High-Signal Facts - Prioritize concrete details: names, roles, actions, dates, amounts, locations, ownership, conflicts. - Ignore trivial or repeated facts. 3. Generate Summary - Write 1–2 sentences in clear, neutral prose. - Make sure the summary is directly grounded in the provided text. - Do not add any commentary about uncertainty, importance, or significance unless explicitly stated in the input. [Input] Text: {community_linearized_text} [Output] Return ONLY the final summary as plain text, in 1–2 sentences total. Do not include any labels, headings, metadata, bullet points, or analysis.

Figure 7: Community Summary Generation Prompt (Π_s) used by the KGSum Agent to produce concise summaries for individual KG communities.

Appendix B Detailed Analysis of KGSum Agent

This section provides comprehensive analysis of the KGSum Agent’s key design decisions. We validate the effectiveness of the Leiden algorithm for community detection and examine the impact of contrastive relation extraction on summary quality. These experiments demonstrate that both architectural choices contribute substantially to the agent’s performance.

B.1 Leiden Algorithm Performance Validation

We validate the Leiden algorithm choice through comparative evaluation against Standard Louvain and a no-community-detection baseline on the Multi-News dataset. Table 9 presents results for both structural quality (Modularity) and downstream summarization performance (ROUGE scores).

Table 9: Comparison of community detection algorithms on Multi-News

Configuration	Modularity	ROUGE-1	ROUGE-2	ROUGE-L
No Community (Baseline)	0.0000	0.4955	0.2183	0.2241
Standard Louvain	0.5470	0.5281	0.2314	0.2655
Leiden (Our Approach)	0.5512	0.5497	0.2490	0.2940

Leiden achieves superior modularity (0.5512 vs 0.5470 for Louvain), indicating more robust community structure. Moreover, Leiden translates this structural advantage into downstream performance gains of 4.1 percent in ROUGE-1, 7.6 percent in ROUGE-2, and 8.8 percent in ROUGE-L over Standard Louvain. Therefore, the quality of community detection directly affects summarization effectiveness. By producing thematically

[Instruction]
 You are a multilingual, metadata-aware summary synthesizer. You generate higher-level fused summaries by integrating multiple community-level summaries from a knowledge graph along with their metadata.

[Objective]
 Given:
 1. A list of community summaries.
 2. For each community, metadata including:
 - "centrality": numeric value in [0.0, 1.0] indicating structural importance.
 - "has_contrasts": boolean indicating whether this community contains internal conflict, disagreement, or contradictory claims.

Your task is to produce a single cohesive summary (3–5 sentences) that:
 - Emphasizes the most central communities.
 - Describes the main facts and relationships across communities.
 - Explicitly acknowledges conflicts where "has_contrasts" is true.

[CHAIN OF THOUGHT]
 1. Rank and Interpret
 - Identify which communities are structurally dominant (very high centrality), supporting (mid centrality), and peripheral (low centrality).
 2. Extract Key Content
 - From high-centrality communities, capture core actors, actions, relationships, numbers, and outcomes.
 - From mid-centrality communities, capture context, consequences, or implications that clarify or enrich the main narrative.
 - From any community with has_contrasts = true, note the presence of disagreement, dispute, or conflicting accounts.
 3. Compose Fused Summary
 - Front-load the most central/global information.
 - Integrate supporting details after the core.
 - Explicitly acknowledge conflict or disagreement if has_contrasts = true in any community.
 - Do not reference "centrality", "metadata", "communities", or the existence of summarization steps; speak as if summarizing a single situation.

[Input]
 communities: [
 {
 "summary": "<community-level summary text>",
 "metadata": {
 "centrality": <number between 0.0 and 1.0>,
 "has_contrasts": <true | false>
 }
 },
 ...
]

[Output]
 Return ONLY one fused summary in plain text, 3–5 sentences total, written in inverted pyramid style as described above. Do not include any labels, headings, JSON, bullet points, or analysis.

Figure 8: Meta-Aware Fusion Prompt (Π_f) used by the KGSum Agent to synthesize a final summary from community summaries, guided by metadata.

<i>[Instruction]</i> You are a multilingual, multi-document abstractive summarization model. You generate a single, coherent summary that captures the main themes across a collection of related documents. Write in the same language as the majority of the input documents unless otherwise specified. <i>[Objective]</i> Given: 1. A document collection D (possibly spanning multiple sources, sections, or time periods). Your task is to produce an initial abstractive summary that: - captures the central topics, key entities, events, and relationships present across D, - preserves important numbers, dates, and causal links when they are salient, - is concise but comprehensive enough to serve as a foundation for later refinement stages. <i>[CHAIN OF THOUGHT]</i> 1. Multi-Document Scan - Identify recurring topics, main entities, core events, and major claims. 2. Theme Aggregation - Group related information across documents into a small set of main themes. - Resolve obvious redundancies: if multiple documents repeat the same point, include it once in the summary. 3. Initial Summary Composition - Write a coherent abstractive summary that: - Gives a high-level overview first, then integrates the most important details. - Connects key entities and events with clear causal or temporal structure where evident. <i>[Input]</i> Documents (D) <i>[Output]</i> <i>[INITIAL SUMMARY]</i> Return ONLY the initial summary text as plain text, with no extra labels, headings, bullet points, or commentary.

Figure 9: Abstractor Agent Prompt Template for Initial Summary Generation (Π_{init}), guiding the LLM to produce a first-pass abstractive summary from the document set.

coherent communities, Leiden enables meta-aware prompts to operate more effectively, improving content selection and semantic coherence.

B.2 Impact of Contrastive Relation Extraction

To demonstrate the importance of contrastive relation extraction in the KGSum Agent, we conduct isolated ablation experiments comparing performance with and without contrastive relations on Multi-News and VN-MDS datasets using only the standalone KGSum agent. We evaluate two configurations: Without Contrastive extracts only standard relations (e.g., “related to”, “part of”), while With Contrastive additionally identifies contrastive relations (e.g., “contradicts”, “differs from”) using prompt Π_{rel} (Appendix A).

Table 10 presents performance comparison. The With Contrastive configuration achieves substantial gains on Multi-News: 15.6 percent improvement in ROUGE-1, 100.0 percent in ROUGE-2, and 25.0 percent in ROUGE-L. On VN-MDS, gains are consistent: 5.0 percent in ROUGE-1, 5.2 percent in ROUGE-2, and 5.1 percent in ROUGE-L. The larger gains on Multi-News suggest that contrastive information is particularly valuable for English news articles, where multiple sources often present conflicting perspectives.

Table 10: Effect of contrastive relation extraction on standalone KGSum

Configuration	Multi-News			VN-MDS		
	R1	R2	RL	R1	R2	RL
Without Contrastive	0.32	0.04	0.12	0.5689	0.0873	0.2494
With Contrastive	0.37	0.08	0.15	0.5976	0.0918	0.2622

To illustrate how contrastive relations improve quality, we examine Sample 3 from Multi-News discussing Microsoft’s Nokia acquisition. Figure 12 compares summaries with and without contrastive extraction. The contrastive-aware summary demonstrates three critical improvements. **First**, it explicitly contrasts Nokia’s historical dominance (35 percent market share) with its decline, providing context for the deal’s strategic

[Instruction]
 You are a multilingual senior editorial agent responsible for high-quality refinement of multi-source summaries. Your job is not to generate new claims, but to improve an existing summary using structured feedback.

[Objective]
 Given:
 1. A current summary draft.
 2. A structured feedback object generated by an external review function (FMDS).

Your task is to produce an improved summary that:

- Incorporates required factual details (e.g. missing figures, actors, timelines) when they are explicitly provided in the feedback.
- Resolves any flagged inconsistencies or ambiguities.
- Removes redundancy and repetitive phrasing.
- Improves clarity, coherence, and flow.

Constraints:

- You MUST follow the requested edits and priorities in the feedback object.
- You MUST NOT introduce facts that are not present in either the current summary or the feedback object.
- You MUST output a clean, single summary in natural prose. No analysis of what you changed.

[CHAIN OF THOUGHT]

1. Parse Feedback
 - Read the feedback object.
 - Identify explicit requested actions, such as:
 - "Incorporate more specific figures"
 - "Clarify X vs Y"
 - "Remove redundant phrasing about Z"
 - "Emphasize the following point: ..."
 - Note any factual corrections or conflict resolution instructions (e.g. "Use 12.3M, not 10M").
2. Plan Revision
 - Update the current summary to include missing concrete details that the feedback explicitly supplies.
 - Resolve contradictions by preferring the corrected fact(s) given in the feedback.
 - Merge repetitive or fragmented sentences for smoother flow.
 - Preserve all core meaning from the current summary unless the feedback says to downplay or remove something.
3. Rewrite Summary
 - Produce a refined version that:
 - Is factually consistent with the corrected/augmented details.
 - Avoids repetition.
 - Reads as a single, coherent narrative.
 - Do NOT mention that you received feedback, and do NOT describe edits.
 - Do NOT include bullet points unless the current summary was already in bullet form and the feedback does not instruct you to remove that structure.

[Input]
 Current Summary: {current_summary_text}
 Feedback Object: {feedback_object_from_FMDS}

The Feedback Object may include:

- Required factual insertions.
- Requested tone/structure adjustments.
- Specific issues to fix (redundancy, ambiguity, missing attribution).
- Phrasing constraints (e.g. "avoid speculative language", "be more specific about timelines").

[Output]
[IMPROVED SUMMARY]
 Return ONLY the revised summary text, incorporating the feedback.
 Do not include any meta commentary, justification, labels, or diagnostics.

Figure 10: Abstractor Agent Prompt Template for Iterative Refinement (Π_{ref}), guiding the LLM to improve summaries based on structured feedback.

[Instruction]
 You are a multilingual, fact-grounded summarization model. Write concise, neutral summaries in {output_language} (default: match input language). Do NOT add information not present in the source.

[Objective]
 Given a single source text, produce a summary that:
 - captures the main point, key entities, numbers, and causal links,
 - avoids opinion, hype, and speculation,
 - preserves critical nuance and avoids hallucination.

[CHAIN OF THOUGHT]
 1. Identify Core: Determine the central topic, key actors, actions, outcomes, and numbers.
 2. De-duplicate: Merge overlapping details, drop trivia and side-notes.
 3. Compose: Write in clear, neutral prose; no bullets, no headings.
 - If conflicting claims exist, briefly note the disagreement without resolving it beyond what the text states.

[Input]
 Text: {source_text}

[Output]
 Return ONLY the final summary as plain text, with no extra labels or commentary.

Figure 11: Few-Shot Prompt Template (Π_{few}) used to compare between performance of LLMs with our MoA framework and standalone LLM summarization.

necessity. **Second**, it frames Microsoft’s 15 percent target as “lackluster” relative to Apple and Google, capturing analyst skepticism. **Third**, it emphasizes the fundamental tension between Microsoft’s hardware ambitions and the shifting mobile ecosystem, rather than simply listing concerns. Therefore, contrastive relations enable the KGSum agent to capture conflicting perspectives, temporal contrasts, and competitive tensions that would otherwise be lost, explaining the substantial ROUGE improvements in Table 10.

[Summary With Contrastive Relations]
 In 2014, Microsoft solidified its commitment to the mobile market by acquiring Nokia’s phone division for \$5.44 billion, which included \$1.65 billion allocated specifically for Nokia’s patent portfolio. **This strategic move was essential as Nokia, which had once commanded over 35% of the smartphone market share in 2003, struggled with innovation and operating profits of merely \$5.48 billion, diminishing significantly by the time of acquisition.** Analysts projected Microsoft aimed for a modest 15% share of the global smartphone market by 2018, **a target considered lackluster compared to the competitive landscapes dominated by Apple and Google.** Despite its ambitions, **Microsoft faced substantial hurdles regarding brand integration and adapting to an ecosystem shifting rapidly towards mobile services, raising concerns about the unification of its broader product portfolio, which includes Windows and Office.** Ultimately, **these challenges not only highlighted the fierce competitive dynamics but also underscored the critical need for Microsoft to streamline its hardware and software strategies to maintain relevance in an evolving industry landscape.**

[Summary Without Contrastive Relations]
 On September 2, 2013, Microsoft acquired Nokia’s Devices and Services division for approximately \$7.2 billion, including \$1.65 billion for Nokia’s patents, with aspirations to capture 15% of the global smartphone market and generate over \$45 billion in revenue by 2018. However, analysts remain skeptical about these ambitious plans due to Nokia’s drastic decline from holding over 35% of the global phone market in 2003 to struggling with its Lumia series, revealing a failure to innovate effectively. The acquisition aims to enhance Microsoft’s strategic integration with hardware, especially in emerging markets where Nokia’s brand, particularly through low-cost models like the Asha series, retains influence. Despite this, concerns remain about potential conflicts with existing hardware manufacturers, echoing worries regarding leadership decisions within Nokia and market miscalculations that resulted in a significant disconnect between aspirations and market realities.

Figure 12: Comparison of summaries generated by KGSum Agent without and with contrastive relation extraction for Sample 3 from Multi-News. The contrastive-aware summary captures nuanced tensions and conflicting perspectives. Key improvements are highlighted in red.

B.3 Error Propagation Analysis in Hybrid Extraction

The KGSum agent employs a hybrid extraction approach combining spaCy for entity recognition, rule-based filtering, and LLM reasoning for relation classification. To address concerns about error propagation from noisy KG components through community detection to final summaries, we conduct controlled noise injection experiments on the Multi-News dataset. We evaluate three scenarios: Clean Baseline (0 percent entity/relation noise), Moderate Noise (20 percent entity, 30 percent relation), and High Noise (40 percent entity, 50

percent relation). Noise is randomly injected as spurious entities and incorrect relations before community detection. We measure KG Noise Score (proportion of injected noise retained), Noise Amplification (noise ratio in summaries to input), and ROUGE performance.

Table 11 presents results across all scenarios. The hybrid approach demonstrates strong robustness. KG Noise Score remains 0.0000 across all conditions, indicating the hybrid filtering successfully rejects injected noise before it enters the graph structure. The combination of spaCy statistical recognition, rule-based validation, and LLM reasoning provides multiple verification layers. Moreover, Noise Amplification is consistently 0.0000, demonstrating that the multi-stage architecture prevents error cascading even if minimal noise enters. ROUGE scores show graceful degradation: ROUGE-1 decreases from 37.0 (Clean) to 36.8 (Moderate) to 36.5 (High), ROUGE-2 from 4.0 to 3.9 to 3.8, and ROUGE-L from 12.0 to 11.9 to 11.7. The minimal performance degradation (less than 2 percent across all metrics) despite substantial injected noise (up to 40 percent entity and 50 percent relation noise) validates the robustness of the hybrid extraction architecture. Therefore, the hybrid extraction architecture acts as a robust filtering mechanism, preventing noisy components from contaminating downstream processes and ensuring reliable performance for real-world deployment.

Table 11: Robustness of KGSum under controlled noise on Multi-News

Scenario	KG Noise	Amplification	R1	R2	RL
Clean (0% E, 0% R)	0.0000	0.0000	37.0 (± 1.2)	4.0	12.0
Moderate (20% E, 30% R)	0.0000	0.0000	36.8 (± 1.3)	3.9	11.9
High (40% E, 50% R)	0.0000	0.0000	36.5 (± 1.4)	3.8	11.7

Note: E = Entity noise, R = Relation noise. Standard deviations shown for R1.

Appendix C Detailed Dataset Descriptions

C.1 Dataset Overview

We evaluate MoA on four diverse MDS datasets spanning multiple domains and languages. Multi-News [Fabbri et al., 2019] consists of news articles from multiple sources covering the same event, requiring synthesis of potentially conflicting perspectives and redundant information. Multi-XScience [Lu et al., 2020] comprises scientific papers with related work sections, requiring capture of technical terminology, citation relationships, and methodological comparisons. VN-MDS¹¹ contains Vietnamese news articles from multiple outlets, testing cross-lingual applicability and cultural context handling. ViMs [Tran et al., 2020] includes diverse Vietnamese documents spanning multiple genres, evaluating robustness to varied writing styles and topical domains. This evaluation demonstrates our framework’s capability to handle complex inter-document relationships across diverse domains and languages.

For all datasets, we apply consistent preprocessing to ensure clean LLM input. Input documents are segmented into sentences using language-specific tokenizers: spaCy for English datasets and Vietnamese-appropriate tokenizers for Vietnamese datasets. Special characters including excessive punctuation, non-standard symbols, and formatting artifacts are removed to prevent tokenization errors. The training set is used solely for prompt optimization, as MoA is a training-free framework. Test splits are reserved for evaluation to ensure fair comparison with published baselines. Therefore, all reported results reflect true generalization performance on held-out test data.

C.2 Comparative Complexity Analysis

To understand performance differences between English and Vietnamese datasets, we conduct systematic complexity analysis. Vietnamese test sets contain substantially fewer samples (29 to 45) compared to English datasets (5,000 plus), making direct comparison challenging. Therefore, we sample 100 instances from training sets of all four datasets to enable fair cross-dataset complexity comparison. We evaluate three key dimensions: Type-Token Ratio or TTR (lexical diversity, higher is more complex), N-gram Overlap (content redundancy, lower is more complex), and Extractiveness (proportion of summary directly copied from source, lower is more complex). Table 12 presents comprehensive metrics. **Bold red** indicates highest complexity, **bold black** indicates second-highest complexity within each metric category.

¹¹<https://github.com/lupanh/VietnameseMDS>

Table 12: Comparative dataset complexity metrics

Complexity Metric	Multi-News	Multi-XScience	VN-MDS	ViMs
<i>Lexical Diversity (higher = more challenging)</i>				
Document TTR	0.1135	0.1106	0.0314	0.0200
Summary TTR	0.2339	0.2246	0.1204	0.1081
<i>Content Redundancy (lower = more challenging)</i>				
Bigram overlap	0.0445	0.0273	0.1253	0.1736
Trigram overlap	0.0174	0.0083	0.0772	0.1113
<i>Summary Characteristics (lower extractiveness = more challenging)</i>				
Avg. extractiveness	0.2760	0.1075	0.9267	0.9601
Avg. novel content	0.7240	0.8925	0.0733	0.0399

The analysis reveals systematic complexity differences strongly favoring English datasets. English datasets demonstrate substantially higher lexical diversity with document TTR 4.4 times higher than Vietnamese (0.1121 versus 0.0257 average), requiring models to handle diverse linguistic expressions and paraphrasing capabilities. Vietnamese datasets exhibit significantly higher content redundancy with bigram overlap 4.2 times higher (0.1495 versus 0.0359 average) and trigram overlap 7.4 times higher (0.0942 versus 0.0128 average), simplifying information synthesis through multiple overlapping formulations. Vietnamese summaries are highly extractive (94.3 percent versus 19.2 percent for English), inflating ROUGE scores through trivial exact n-gram matching rather than demanding abstractive synthesis. Therefore, higher absolute ROUGE scores on Vietnamese datasets reflect inherently less challenging evaluation conditions rather than superior model capabilities for Vietnamese language processing.

Appendix D Hyperparameter Configuration

Table 13 presents complete hyperparameter configuration for the MoA framework. All hyperparameter values are determined through systematic prompt optimization on the training set, balancing performance quality with computational efficiency. The Extractor agent employs sentence scoring weight $\lambda = 0.7$ to balance lexical and positional importance, with top-k sentence proportion α targeting approximately 15 percent of source length for sufficient context while maintaining computational efficiency. Redundancy threshold $\theta_r = 0.9$ filters near-duplicate sentences to reduce information redundancy. The KGSUM agent uses entity similarity threshold $\theta_s = 0.85$ for entity resolution, relation confidence threshold $\tau_r = 0.6$ for relation extraction quality, and selects $k = 5$ top communities with minimum size 3 and maximum cluster size 50 to balance coverage with computational tractability. The Leiden resolution parameter is set to 1.0 for standard modularity optimization. The Abstractor agent performs up to $T = 2$ refinement iterations with quality threshold $\tau_q^{\text{Abs}} = 0.7$ for triggering refinement, balancing quality improvement with API cost efficiency.

We set LLM top-p sampling to 0.8 for all agents to balance output diversity with response stability. This value ensures sufficient creativity for summarization while maintaining consistency across multiple runs. All experiments are conducted on NVIDIA Quadro RTX 8000 GPU with sufficient memory for batch processing and caching. We perform three independent experimental runs for statistical validation, reporting mean and standard deviation to quantify performance variability and ensure reproducibility. Therefore, all reported results reflect averaged performance across multiple runs, demonstrating the stability and reliability of our framework across different random seeds and initialization conditions.

Appendix E API Cost Analysis for Closed-Source Models

To address practical deployment considerations, we analyze API costs for closed-source LLMs across all evaluation datasets. Costs represent actual expenditures from a single experimental run per dataset, calculated based on official API pricing structures from respective providers (OpenAI, Google, xAI). All costs are reported in USD and include regular input tokens, cached input tokens, and output tokens. This analysis enables informed model selection decisions for real-world applications balancing performance and budget constraints.

Table 13: Hyperparameter configuration

Parameter	Component	Value
<i>MoA framework hyperparameters</i>		
Sentence Scoring Weight (λ)	Extractor agent	0.7
Top-k Sentences Proportion (α)	Extractor agent	Target: $\sim 15\%$ of source length
Redundancy Threshold (θ_r)	Extractor agent	0.9
Entity Similarity Threshold (θ_s)	KGSum agent	0.85
Relation Confidence Threshold (τ_r)	KGSum agent	0.6
Number of Top Communities (k)	KGSum agent	5
Minimum Community Size	KGSum agent	3
Maximum Cluster Size	KGSum agent	50
Leiden Resolution Parameter	KGSum agent	1.0
Max Refinement Iterations (T)	Abstractor agent	2
Quality for Refinement (τ_q^{Abs})	Abstractor agent	0.7
<i>LLM configuration</i>		
LLM Top-p	All agents	0.8
<i>Computational Resources</i>		
GPU	Experimentation	NVIDIA Quadro RTX 8000
Number of Runs	Statistical Validation	3 independent runs (averaged results)

*Note:**Prompt design strategies are detailed in Appendix A.*

Table 14 presents comprehensive API cost breakdown across four closed-source models. The analysis reveals substantial cost variations. GPT-4.1-mini achieves superior performance but incurs the highest costs across all datasets: \$94.55 on Multi-News (\$0.0168 per sample), \$58.39 on Multi-XScience (\$0.0115 per sample), \$0.20 on VN-MDS (\$0.0070 per sample), and \$0.51 on ViMs (\$0.0113 per sample). In contrast, Gemini-1.5-flash-8b offers the most cost-effective solution with costs ranging from \$0.02 on VN-MDS to \$10.07 on Multi-News, representing 5.6 to 10.6 times lower costs than GPT-4.1-mini while maintaining competitive performance. GPT-4o-mini and Grok-3-mini provide intermediate cost-performance tradeoffs. Moreover, input token caching reduces costs by 4.4 to 6.3 percent across models, demonstrating efficiency gains from prompt reuse in multi-document processing. Therefore, model selection should consider the performance-cost tradeoff based on application requirements and budget constraints.

Appendix F Statistical Significance Validation

To validate the reliability of our performance improvements, we conduct dataset-level statistical validation using the sign test. This non-parametric method evaluates whether MoA systematically outperforms published baselines across 12 independent dataset-metric combinations derived from 4 evaluation datasets (Multi-News, Multi-XScience, VN-MDS, ViMs) and 3 ROUGE metrics (R1, R2, RL). For each combination, we compare MoA against the best published baseline as reported in Table 4 and Table 5. The sign test counts comparisons where MoA wins. Under the null hypothesis of equal performance, the expected win proportion is 0.5. Therefore, observing significantly higher win rates provides evidence of systematic superiority.

Table 15 presents validation results. MoA achieves superior performance in all 12 dataset-metric combinations (100 percent win rate). The one-tailed binomial test yields $p < 0.001$ for observing 12 wins out of 12 trials, providing conclusive evidence that MoA’s superior performance reflects genuine systematic improvements rather than random chance. Moreover, the consistency of wins across diverse content types (news articles, scientific papers, diverse documents) and languages (English, Vietnamese) demonstrates robustness and generalizability.

Beyond statistical significance, we examine practical significance through effect sizes. Improvements range from 4.2 percent (ViMs RL) to 48.2 percent (Multi-News R2), with median improvement of 15.3 percent across all combinations. In particular, substantial gains on challenging English benchmarks (Multi-News, Multi-XScience) demonstrate meaningful performance enhancements for real-world applications. Furthermore, consistent improvements across both training-free and supervised baselines validate that our architectural coordination addresses fundamental limitations in existing MDS methodologies, establishing MoA as a reliable and generalizable solution.

Table 14: API cost analysis for closed-source LLMs

Model	Reg. In (\$)	Cache In (\$)	Out (\$)	Total (\$)	Per Sample (\$)
Multi-News (5,622 samples)					
GPT-4.1-mini	68.42	4.28	21.86	94.55	0.0168
GPT-4o-mini	25.66	3.21	8.20	37.06	0.0066
Gemini-1.5-flash-8b	6.41	1.60	2.05	10.07	0.0018
Grok-3-mini	42.76	3.21	6.83	52.80	0.0094
Multi-XScience (5,093 samples)					
GPT-4.1-mini	36.32	2.27	19.80	58.39	0.0115
GPT-4o-mini	13.62	1.70	7.43	22.75	0.0045
Gemini-1.5-flash-8b	3.40	0.85	1.86	6.11	0.0012
Grok-3-mini	22.70	1.70	6.19	30.59	0.0060
VN-MDS (29 samples)					
GPT-4.1-mini	0.09	0.01	0.11	0.20	0.0070
GPT-4o-mini	0.03	0.00	0.04	0.08	0.0027
Gemini-1.5-flash-8b	0.01	0.00	0.01	0.02	0.0007
Grok-3-mini	0.05	0.00	0.04	0.09	0.0032
ViMs (45 samples)					
GPT-4.1-mini	0.31	0.02	0.17	0.51	0.0113
GPT-4o-mini	0.12	0.01	0.07	0.20	0.0044
Gemini-1.5-flash-8b	0.03	0.01	0.02	0.05	0.0012
Grok-3-mini	0.19	0.01	0.05	0.26	0.0059

Note: Reg. In = Regular Input tokens, Cache In = Cached Input tokens, Out = Output tokens. **Bold** indicates highest cost within each dataset.

Table 15: Statistical validation against best published baselines

Dataset	Metric	Best Baseline	MoA	Win?
Multi-News	R1	0.4570 (Centrum)	0.5497	✓
Multi-News	R2	0.1680 (Centrum)	0.2490	✓
Multi-News	RL	0.2320 (Centrum)	0.2940	✓
Multi-XScience	R1	0.3179 (GLIMMER-TTR)	0.3666	✓
Multi-XScience	R2	0.0481 (GLIMMER-TTR)	0.0564	✓
Multi-XScience	RL	0.1671 (GLIMMER-TTR)	0.1973	✓
VN-MDS	R1	0.7676 (LatVisNewshead)	0.8011	✓
VN-MDS	R2	0.5021 (LatVisNewshead)	0.5232	✓
VN-MDS	RL	0.4739 (LatVisNewshead)	0.4829	✓
ViMs	R1	0.7898 (LatVisNewshead)	0.8026	✓
ViMs	R2	0.5508 (LatVisNewshead)	0.5557	✓
ViMs	RL	0.4800 (LatVisNewshead)	0.4904	✓
Total			12/12	100%

Appendix G Human Evaluation Interface and Guidelines

This appendix describes our human evaluation methodology for assessing summary quality across multiple systems. Figure 13 presents the annotation interface used for comparative evaluation. For each document cluster, participants evaluate three summaries labeled Summary A, Summary B, and Summary C, where the mapping between labels and actual systems is hidden to prevent bias. Participants rate each summary on four quality criteria using a 1 to 5 scale: (1) Grammaticality assesses whether sentences are grammatical, fluent, and easy to read without obvious grammar or formatting errors; (2) Non-redundancy evaluates whether the summary avoids repeating the same information unnecessarily; (3) Referential Clarity examines whether it is clear who or what each mention refers to; (4) Structure and Coherence measures whether the summary is well organized and logically flowing rather than disconnected sentences. The rating scale ranges from 1 (Very Poor) to 5 (Very Good), with intermediate levels at 2 (Poor), 3 (Barely Acceptable), and 4 (Good).

For each topic, you will be shown three summaries, labeled **Summary A**, **Summary B**, and **Summary C**. The mapping between A/B/C and actual systems is hidden on purpose.

You will rate each system on five criteria, using a 1–5 scale where:

1 = Very Poor

2 = Poor

3 = Barely Acceptable

4 = Good

5 = Very Good

Criteria definitions:

1. **Grammaticality:** Are the sentences grammatical, fluent, and easy to read? Are there obvious grammar or formatting errors?
2. **Non-redundancy:** Does the summary avoid repeating the same information unnecessarily?
3. **Referential Clarity:** Is it clear who or what each mention (name, noun phrase, pronoun like “he”, “they”, “it”) refers to?
4. **Structure & Coherence:** Is the summary well organized and logically flowing, instead of being just a bag of disconnected sentences?

How to answer each question in this form:

For every question, please give scores for A, B, and C in a single line using this format:

A-x, B-y, C-z

For example:

A-4, B-2, C-5

Figure 13: Human Evaluation Interface used for assessing summary quality based on Grammaticality, Non-redundancy, Referential Clarity, and Structure and Coherence criteria.

Participants receive source documents and three anonymized summaries in randomly ordered presentation for each evaluation task. They independently rate each summary on all four criteria after reading source documents to understand the content. The interface requires participants to provide scores for all three summaries (A, B, C) in a single line using the format A-x, B-y, C-z (e.g., A-4, B-2, C-5). Moreover, we implement quality control through comprehensive training materials explaining each criterion with concrete examples, attention check questions to identify inattentive participants, and inter-annotator agreement computation to verify consistency. Participants showing low agreement with others or failing attention checks are excluded from final analysis. Therefore, this rigorous protocol ensures that reported results reflect genuine quality assessments rather than random ratings, providing reliable validation of our framework’s performance improvements.

Appendix H Impact of Cluster Size on MoA Performance

To evaluate the robustness of the MoA framework under varying document quantities, we conduct a focused scaling study on Multi-News. We systematically compare performance across three cluster size configurations: Small (3 documents per cluster), Medium (5 documents per cluster), and Large (10 documents per cluster). For each configuration, we evaluate 10 randomly sampled clusters with 3 independent runs per sample, reporting mean and standard deviation across 3 runs. All experimental conditions remain constant (GPT-4.1-mini backend, prompt templates, token budget, filtering thresholds, fusion strategies) with only cluster size varying to isolate its effect on framework performance.

Table 16 presents scaling results. The framework demonstrates graceful degradation as cluster size increases. Small (3 documents) achieves optimal performance with 60.2 percent ROUGE-1, 28.1 percent ROUGE-2, and 31.8 percent ROUGE-L. Medium (5 documents) shows slight degradation with 55.4 percent ROUGE-1 (4.8 points decrease, 7.97 percent relative), 24.9 percent ROUGE-2 (3.2 points decrease, 11.39 percent relative), and 29.4 percent ROUGE-L (2.4 points decrease, 7.55 percent relative). Large (10 documents) exhibits more substantial degradation with 49.2 percent ROUGE-1 (11.0 points decrease from Small, 18.27 percent relative), 21.8 percent ROUGE-2 (6.3 points decrease, 22.42 percent relative), and 27.2 percent ROUGE-L (4.6 points decrease, 14.47 percent relative).

Table 16: Scaling study on Multi-News by cluster size

Cluster size	R1	R2	RL
Small (3)	60.2±0.4	28.1±0.3	31.8±0.3
Medium (5)	55.4±0.3	24.9±0.3	29.4±0.2
Large (10)	49.2±0.5	21.8±0.4	27.2±0.3

These results demonstrate three critical insights. First, the framework maintains acceptable performance even at 10 documents per cluster, with ROUGE-L degradation limited to 14.47 percent relative to optimal conditions. Second, the degradation pattern is relatively stable from Medium to Large configurations, suggesting architectural robustness beyond typical dataset scales. Third, the performance-cost tradeoff enables informed deployment decisions: applications prioritizing quality can use smaller clusters (3 to 5 documents), while cost-sensitive deployments can scale to larger clusters (10 documents) with graceful degradation. Therefore, the MoA framework demonstrates robust scalability across document quantities, validating its applicability for real-world multi-document summarization scenarios with varying cluster sizes.